

CIS 6270

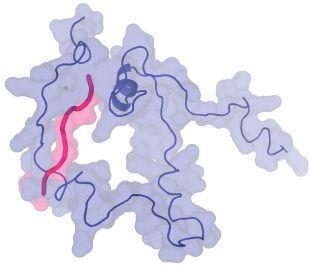
Module 5: Discrete Flow Matching

Lecture 5.2: Correctors, Kinetic-Optimal Paths, and Edit Flows

Thu, Oct 29, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Average the mixture-path velocity over the posterior

Average the mixture-path velocity over the posterior

Substitute the conditional velocity and average:

$$\boxed{u_t^i(y^i, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(y^i \mid x) - \delta_{x^i}(y^i) \right]}$$

Average the mixture-path velocity over the posterior

Substitute the conditional velocity and average:

$$\boxed{u_t^i(y^i, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(y^i \mid x) - \delta_{x^i}(y^i) \right]}$$

The marginal velocity at position i from the sequence x to the token y^i equals $\dot{\kappa}_t$ divided by one minus κ_t, times the denoiser probability $p_{1|t}(y^i \mid x)$ that the data token is y^i, minus the indicator that y^i is the current token x^i.

Average the mixture-path velocity over the posterior

Substitute the conditional velocity and average:

$$u_t^i(y^i, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(y^i \mid x) - \delta_{x^i}(y^i) \right]$$

The marginal velocity at position i from the sequence x to the token y^i equals $\dot{\kappa}_t$ divided by one minus κ_t, times the denoiser probability $p_{1|t}(y^i \mid x)$ that the data token is y^i, minus the indicator that y^i is the current token x^i.

The conditional velocity depends on the pair only through the data token.

Average the mixture-path velocity over the posterior

Substitute the conditional velocity and average:

$$u_t^i(y^i, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(y^i \mid x) - \delta_{x^i}(y^i) \right]$$

The marginal velocity at position i from the sequence x to the token y^i equals $\dot{\kappa}_t$ divided by one minus κ_t, times the denoiser probability $p_{1|t}(y^i \mid x)$ that the data token is y^i, minus the indicator that y^i is the current token x^i.

The conditional velocity depends on the pair only through the data token. We can therefore sum over that token alone.

Average the mixture-path velocity over the posterior

Substitute the conditional velocity and average:

$$u_t^i(y^i, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(y^i \mid x) - \delta_{x^i}(y^i) \right]$$

The marginal velocity at position i from the sequence x to the token y^i equals $\dot{\kappa}_t$ divided by one minus κ_t, times the denoiser probability $p_{1|t}(y^i \mid x)$ that the data token is y^i, minus the indicator that y^i is the current token x^i.

When several clean tokens could explain the current sequence, the outgoing rate is split according to that uncertainty.

The denoiser

The denoiser

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\, x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x)$$

The denoiser

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\, x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x)$$

The denoiser probability of the token y^i at position i is the posterior probability, given the current sequence x, that the data token x^i_1 equals y^i.

The denoiser

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\, x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x)$$

The denoiser probability of the token y^i at position i is the posterior probability, given the current sequence x, that the data token x^i_1 equals y^i.

As in discrete diffusion, we train the network to predict the data token.

The denoiser

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\, x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x)$$

The denoiser probability of the token y^i at position i is the posterior probability, given the current sequence x, that the data token x^i_1 equals y^i.

As in discrete diffusion, we train the network to predict the data token. We then use the schedule to convert that prediction into a velocity.

The denoiser

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\, x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x)$$

The denoiser probability of the token y^i at position i is the posterior probability, given the current sequence x, that the data token x^i_1 equals y^i.

As in discrete diffusion, we train the network to predict the data token. We then use the schedule to convert that prediction into a velocity.

With one network call, we predict a token distribution at every position.

The denoiser

$$p_{1|t}(y^i \mid x) = \sum_{x_0,\, x_1} \delta_{x_1^i}(y^i)\; p_t(x_0, x_1 \mid x)$$

The denoiser probability of the token y^i at position i is the posterior probability, given the current sequence x, that the data token x^i_1 equals y^i.

As in discrete diffusion, we train the network to predict the data token. We then use the schedule to convert that prediction into a velocity.

With one network call, we predict a token distribution at every position. To compute the velocity, we subtract the point mass on the current token and multiply by the schedule factor.

Let’s average the possible jumps from G

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1).

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa_t}/(1 - \kappa_t)$ is 2.

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ is 2.

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2)$$

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ is 2.

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2)$$

The velocity equals 2 times the denoiser probabilities minus the indicator of G, which gives rates 0.2, 1.2, and 0.2 toward A, C, and T, and –1.6 on the diagonal for G.

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ is 2.

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2)$$

One Euler step with h = 0.1 gives $\delta_{\mathrm{G}} + 0.1\, u_t(\cdot, x) = (0.02,\ 0.12,\ 0.84,\ 0.02)$

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ is 2.

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2)$$

One Euler step with h = 0.1 gives $\delta_{\mathrm{G}} + 0.1\, u_t(\cdot, x) = (0.02,\ 0.12,\ 0.84,\ 0.02)$

After one step of size 0.1, the position becomes A with probability 0.02, C with probability 0.12, stays G with probability 0.84, and becomes T with probability 0.02.

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ is 2.

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2)$$

One Euler step with h = 0.1 gives $\delta_{\mathrm{G}} + 0.1\, u_t(\cdot, x) = (0.02,\ 0.12,\ 0.84,\ 0.02)$

The four probabilities sum to one.

Let’s average the possible jumps from G

The current token is G. The denoiser probabilities for (A, C, G, T) are (0.1, 0.6, 0.2, 0.1). With $\kappa_t = t$ at $t = 0.5$, the schedule factor $\dot{\kappa}_t/(1 - \kappa_t)$ is 2.

$$u_t(\cdot, x) = 2\left[(0.1,\ 0.6,\ 0.2,\ 0.1) - (0,\ 0,\ 1,\ 0)\right] = (0.2,\ 1.2,\ -1.6,\ 0.2)$$

One Euler step with h = 0.1 gives $\delta_{\mathrm{G}} + 0.1\, u_t(\cdot, x) = (0.02,\ 0.12,\ 0.84,\ 0.02)$

The four probabilities sum to one. C is the most likely replacement because it has the highest denoiser probability.

Now rewrite the Euler step at one position

Now rewrite the Euler step at one position

Rearrange the Euler step into a two-part rule:

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1 - \lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\, \dot{\kappa}_t}{1 - \kappa_t}$$

Now rewrite the Euler step at one position

Rearrange the Euler step into a two-part rule:

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1 - \lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\, \dot{\kappa}_t}{1 - \kappa_t}$$

The Euler step distribution at position i equals one minus λ_t times the indicator of the current token x^i, plus λ_t times the denoiser distribution $p_{1|t}(\cdot \mid x)$, where λ_t is h times $\dot{\kappa}_t$ divided by one minus κ_t.

Now rewrite the Euler step at one position

Rearrange the Euler step into a two-part rule:

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1 - \lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\, \dot{\kappa}_t}{1 - \kappa_t}$$

The Euler step distribution at position i equals one minus λ_t times the indicator of the current token x^i, plus λ_t times the denoiser distribution $p_{1|t}(\cdot \mid x)$, where λ_t is h times $\dot{\kappa}_t$ divided by one minus κ_t.

With probability λ_t, redraw the position from the denoiser. Otherwise keep it.

Now rewrite the Euler step at one position

Rearrange the Euler step into a two-part rule:

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1 - \lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\, \dot{\kappa}_t}{1 - \kappa_t}$$

The Euler step distribution at position i equals one minus λ_t times the indicator of the current token x^i, plus λ_t times the denoiser distribution $p_{1|t}(\cdot \mid x)$, where λ_t is h times $\dot{\kappa}_t$ divided by one minus κ_t.

With probability λ_t, redraw the position from the denoiser. Otherwise keep it.

In our example, $\lambda_t = 0.2$.

Now rewrite the Euler step at one position

Rearrange the Euler step into a two-part rule:

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1 - \lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\, \dot{\kappa}_t}{1 - \kappa_t}$$

The Euler step distribution at position i equals one minus λ_t times the indicator of the current token x^i, plus λ_t times the denoiser distribution $p_{1|t}(\cdot \mid x)$, where λ_t is h times $\dot{\kappa}_t$ divided by one minus κ_t.

With probability λ_t, redraw the position from the denoiser. Otherwise keep it.

In our example, $\lambda_t = 0.2$. A redraw returns G with probability 0.2, giving a total probability of $0.84 = 0.8 + 0.2 \times 0.2$ for G.

Now rewrite the Euler step at one position

Rearrange the Euler step into a two-part rule:

$$\delta_{x^i}(\cdot) + h\, u_t^i(\cdot, x) = (1 - \lambda_t)\, \delta_{x^i}(\cdot) + \lambda_t\, p_{1|t}(\cdot \mid x), \qquad \lambda_t = \frac{h\, \dot{\kappa}_t}{1 - \kappa_t}$$

The Euler step distribution at position i equals one minus λ_t times the indicator of the current token x^i, plus λ_t times the denoiser distribution $p_{1|t}(\cdot \mid x)$, where λ_t is h times $\dot{\kappa}_t$ divided by one minus κ_t.

With probability λ_t, redraw the position from the denoiser. Otherwise keep it.

In our example, $\lambda_t = 0.2$. A redraw returns G with probability 0.2, giving a total probability of $0.84 = 0.8 + 0.2 \times 0.2$ for G.

The step is valid as long as $\lambda_t \leq 1$, which limits the step size near t = 1.

Put the continuous and discrete velocities side by side

Put the continuous and discrete velocities side by side

Continuous flow matching: $$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right]$$

Put the continuous and discrete velocities side by side

Continuous flow matching: $$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right]$$

The continuous marginal velocity at x equals $\dot{\kappa}_t$ divided by one minus κ_t, times the posterior mean $\hat{x}_{1|t}(x)$ of the data sample minus the current point x.

Put the continuous and discrete velocities side by side

Continuous flow matching: $$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right]$$

The continuous marginal velocity at x equals $\dot{\kappa}_t$ divided by one minus κ_t, times the posterior mean $\hat{x}_{1|t}(x)$ of the data sample minus the current point x.

Discrete, at each position: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right]$$

Put the continuous and discrete velocities side by side

Continuous flow matching: $$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right]$$

The continuous marginal velocity at x equals $\dot{\kappa}_t$ divided by one minus κ_t, times the posterior mean $\hat{x}_{1|t}(x)$ of the data sample minus the current point x.

Discrete, at each position: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right]$$

The discrete marginal velocity at position i equals the same schedule factor times the posterior distribution $p_{1|t}(\cdot \mid x)$ of the data token minus the point mass on the current token x^i.

Put the continuous and discrete velocities side by side

Continuous flow matching: $$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right]$$

The continuous marginal velocity at x equals $\dot{\kappa}_t$ divided by one minus κ_t, times the posterior mean $\hat{x}_{1|t}(x)$ of the data sample minus the current point x.

Discrete, at each position: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right]$$

The discrete marginal velocity at position i equals the same schedule factor times the posterior distribution $p_{1|t}(\cdot \mid x)$ of the data token minus the point mass on the current token x^i.

For discrete states, we replace the posterior mean with a posterior distribution and the current point with a point mass.

Put the continuous and discrete velocities side by side

Continuous flow matching: $$u_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\hat{x}_{1|t}(x) - x \right]$$

The continuous marginal velocity at x equals $\dot{\kappa}_t$ divided by one minus κ_t, times the posterior mean $\hat{x}_{1|t}(x)$ of the data sample minus the current point x.

Discrete, at each position: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right]$$

The discrete marginal velocity at position i equals the same schedule factor times the posterior distribution $p_{1|t}(\cdot \mid x)$ of the data token minus the point mass on the current token x^i.

For discrete states, we replace the posterior mean with a posterior distribution and the current point with a point mass. This is the correspondence derived in the Discrete Flow Matching paper.

We learn the denoiser with cross-entropy

We learn the denoiser with cross-entropy

We can train the posterior by sampling without evaluating it directly.

We learn the denoiser with cross-entropy

We can train the posterior by sampling without evaluating it directly.

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\ (X_0,X_1)\sim\pi,\ X_t\sim p_t(\cdot|X_0,X_1)} \sum_{i=1}^{N} \log p^{\theta}_{1|t}\big(X_1^i \mid X_t\big)$$

We learn the denoiser with cross-entropy

We can train the posterior by sampling without evaluating it directly.

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\ (X_0,X_1)\sim\pi,\ X_t\sim p_t(\cdot|X_0,X_1)} \sum_{i=1}^{N} \log p^{\theta}_{1|t}\big(X_1^i \mid X_t\big)$$

The loss L(θ) is the negative expectation, over a uniform time t, a pair (X_0, X_1) from the coupling π, and a noisy sequence X_t from the conditional path, of the sum over positions i of the log probability that the network $p^{\theta}_{1|t}$ assigns to the true data token X^i_1 given X_t.

We learn the denoiser with cross-entropy

We can train the posterior by sampling without evaluating it directly.

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\ (X_0,X_1)\sim\pi,\ X_t\sim p_t(\cdot|X_0,X_1)} \sum_{i=1}^{N} \log p^{\theta}_{1|t}\big(X_1^i \mid X_t\big)$$

The loss L(θ) is the negative expectation, over a uniform time t, a pair (X_0, X_1) from the coupling π, and a noisy sequence X_t from the conditional path, of the sum over positions i of the log probability that the network $p^{\theta}_{1|t}$ assigns to the true data token X^i_1 given X_t.

Sample a time, a pair, and a noisy sequence, then score the true data token at every position.

We learn the denoiser with cross-entropy

We can train the posterior by sampling without evaluating it directly.

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\ (X_0,X_1)\sim\pi,\ X_t\sim p_t(\cdot|X_0,X_1)} \sum_{i=1}^{N} \log p^{\theta}_{1|t}\big(X_1^i \mid X_t\big)$$

The loss L(θ) is the negative expectation, over a uniform time t, a pair (X_0, X_1) from the coupling π, and a noisy sequence X_t from the conditional path, of the sum over positions i of the log probability that the network $p^{\theta}_{1|t}$ assigns to the true data token X^i_1 given X_t.

Sample a time, a pair, and a noisy sequence, then score the true data token at every position.

We have neither a schedule weight nor a jump rate in the loss.

We learn the denoiser with cross-entropy

We can train the posterior by sampling without evaluating it directly.

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{t,\ (X_0,X_1)\sim\pi,\ X_t\sim p_t(\cdot|X_0,X_1)} \sum_{i=1}^{N} \log p^{\theta}_{1|t}\big(X_1^i \mid X_t\big)$$

The loss L(θ) is the negative expectation, over a uniform time t, a pair (X_0, X_1) from the coupling π, and a noisy sequence X_t from the conditional path, of the sum over positions i of the log probability that the network $p^{\theta}_{1|t}$ assigns to the true data token X^i_1 given X_t.

Sample a time, a pair, and a noisy sequence, then score the true data token at every position.

We have neither a schedule weight nor a jump rate in the loss. Therefore, we can choose the sampler after training.

Now average over the data token first

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\,\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\,\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

The expected loss at position i equals the expectation over X_t of the cross-entropy between the true denoiser $p_{1|t}(\cdot \mid X_t)$ and the network $p^{\theta}_{1|t}(\cdot \mid X_t)$, because the data token given X_t follows the true denoiser.

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\,\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

$$= \mathbb{E}_{X_t}\Big[H\big(p_{1|t}(\cdot \mid X_t)\big) + \mathrm{KL}\big(p_{1|t}(\cdot \mid X_t) \,\big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\big)\Big]$$

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\,\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

$$= \mathbb{E}_{X_t}\Big[H\big(p_{1|t}(\cdot \mid X_t)\big) + \mathrm{KL}\big(p_{1|t}(\cdot \mid X_t) \,\big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\big)\Big]$$

The expected loss equals the expected entropy H of the true denoiser, which does not depend on θ, plus the expected KL divergence from the true denoiser to the network's prediction.

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

$$= \mathbb{E}_{X_t}\Big[H\big(p_{1|t}(\cdot \mid X_t)\big) + \mathrm{KL}\big(p_{1|t}(\cdot \mid X_t) \,\big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\big)\Big]$$

The expected loss equals the expected entropy H of the true denoiser, which does not depend on θ, plus the expected KL divergence from the true denoiser to the network's prediction.

We minimize the loss when the network equals the true denoiser, because the KL term is then zero.

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

$$= \mathbb{E}_{X_t}\Big[H\big(p_{1|t}(\cdot \mid X_t)\big) + \mathrm{KL}\big(p_{1|t}(\cdot \mid X_t) \,\big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\big)\Big]$$

The expected loss equals the expected entropy H of the true denoiser, which does not depend on θ, plus the expected KL divergence from the true denoiser to the network's prediction.

We minimize the loss when the network equals the true denoiser, because the KL term is then zero.

The minimum is the entropy of the conditional data-token distribution.

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

$$= \mathbb{E}_{X_t}\Big[H\big(p_{1|t}(\cdot \mid X_t)\big) + \mathrm{KL}\big(p_{1|t}(\cdot \mid X_t) \,\big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\big)\Big]$$

The expected loss equals the expected entropy H of the true denoiser, which does not depend on θ, plus the expected KL divergence from the true denoiser to the network's prediction.

We minimize the loss when the network equals the true denoiser, because the KL term is then zero.

The minimum is the entropy of the conditional data-token distribution. In continuous flow matching, the corresponding minimum was the variance of the conditional velocity.

Now average over the data token first

Average over X^i_1 given X_t first:

$$\mathbb{E}\big[-\log p^{\theta}_{1|t}(X^i_1 \mid X_t)\big] = \mathbb{E}_{X_t}\Big[\sum_y p_{1|t}(y \mid X_t)\,\big(-\log p^{\theta}_{1|t}(y \mid X_t)\big)\Big]$$

$$= \mathbb{E}_{X_t}\Big[H\big(p_{1|t}(\cdot \mid X_t)\big) + \mathrm{KL}\big(p_{1|t}(\cdot \mid X_t) \,\big\|\, p^{\theta}_{1|t}(\cdot \mid X_t)\big)\Big]$$

The expected loss equals the expected entropy H of the true denoiser, which does not depend on θ, plus the expected KL divergence from the true denoiser to the network's prediction.

We minimize the loss when the network equals the true denoiser, because the KL term is then zero.

Even a perfect denoiser has positive loss when the noisy sequence is compatible with several clean tokens.

Let’s score one training example

Let’s score one training example

Our noisy input is x_t = ATGA. Its target is x_1 = ACGT.

Let’s score one training example

Our noisy input is x_t = ATGA. Its target is x_1 = ACGT.

Suppose the network gives the true tokens probabilities 0.7, 0.2, 0.6, and 0.1 at the four positions.

Let’s score one training example

Our noisy input is x_t = ATGA. Its target is x_1 = ACGT.

Suppose the network gives the true tokens probabilities 0.7, 0.2, 0.6, and 0.1 at the four positions.

$$-\big(\log 0.7 + \log 0.2 + \log 0.6 + \log 0.1\big) = 0.357 + 1.609 + 0.511 + 2.303 = 4.780$$

Let’s score one training example

Our noisy input is x_t = ATGA. Its target is x_1 = ACGT.

Suppose the network gives the true tokens probabilities 0.7, 0.2, 0.6, and 0.1 at the four positions.

$$-\big(\log 0.7 + \log 0.2 + \log 0.6 + \log 0.1\big) = 0.357 + 1.609 + 0.511 + 2.303 = 4.780$$

The loss for this example is the sum of the negative logs of 0.7, 0.2, 0.6, and 0.1, which is 0.357 plus 1.609 plus 0.511 plus 2.303, or 4.780.

Let’s score one training example

Our noisy input is x_t = ATGA. Its target is x_1 = ACGT.

Suppose the network gives the true tokens probabilities 0.7, 0.2, 0.6, and 0.1 at the four positions.

$$-\big(\log 0.7 + \log 0.2 + \log 0.6 + \log 0.1\big) = 0.357 + 1.609 + 0.511 + 2.303 = 4.780$$

Positions 2 and 4 contain source tokens. We must infer C and T from the rest of the sequence, so these positions contribute the most to the loss.

Let’s score one training example

Our noisy input is x_t = ATGA. Its target is x_1 = ACGT.

Suppose the network gives the true tokens probabilities 0.7, 0.2, 0.6, and 0.1 at the four positions.

$$-\big(\log 0.7 + \log 0.2 + \log 0.6 + \log 0.1\big) = 0.357 + 1.609 + 0.511 + 2.303 = 4.780$$

Positions 2 and 4 contain source tokens. We must infer C and T from the rest of the sequence, so these positions contribute the most to the loss.

Every position contributes, including positions that already show their data tokens.

Take the mask source once more

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1-\kappa_t)^{\#\text{masked}(x)}\ \kappa_t^{\#\text{visible}(x)} \prod_{i\ \text{visible}} \delta_{x_1^i}(x^i)$$

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1 - \kappa_t)^{\#\text{masked}(x)} \; \kappa_t^{\#\text{visible}(x)} \prod_{i \;\text{visible}} \delta_{x_1^i}(x^i)$$

The conditional probability of x given the mask token m and the data sequence x_1 equals one minus κ_t raised to the number of masked positions, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches x_1.

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1 - \kappa_t)^{\#\text{masked}(x)} \; \kappa_t^{\#\text{visible}(x)} \prod_{i \;\text{visible}} \delta_{x_1^i}(x^i)$$

The conditional probability of x given the mask token m and the data sequence x_1 equals one minus κ_t raised to the number of masked positions, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches x_1.

The time factor cancels in Bayes' rule because it is the same for every candidate x_1.

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1-\kappa_t)^{\#\text{masked}(x)}\ \kappa_t^{\#\text{visible}(x)} \prod_{i\ \text{visible}} \delta_{x_1^i}(x^i)$$

The conditional probability of x given the mask token m and the data sequence x_1 equals one minus κ_t raised to the number of masked positions, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches x_1.

The time factor cancels in Bayes' rule because it is the same for every candidate x_1. Thus, $p_{1|t}(\cdot \mid x)$ does not depend on t.

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1 - \kappa_t)^{\#\text{masked}(x)} \; \kappa_t^{\#\text{visible}(x)} \prod_{i \;\text{visible}} \delta_{x_1^i}(x^i)$$

The conditional probability of x given the mask token m and the data sequence x_1 equals one minus κ_t raised to the number of masked positions, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches x_1.

The time factor cancels in Bayes' rule because it is the same for every candidate x_1. Thus, $p_{1|t}(\cdot \mid x)$ does not depend on t.

We derived the same result for MDLM.

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1 - \kappa_t)^{\#\text{masked}(x)} \; \kappa_t^{\#\text{visible}(x)} \prod_{i \;\text{visible}} \delta_{x_1^i}(x^i)$$

The conditional probability of x given the mask token m and the data sequence x_1 equals one minus κ_t raised to the number of masked positions, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches x_1.

The time factor cancels in Bayes' rule because it is the same for every candidate x_1. Thus, $p_{1|t}(\cdot \mid x)$ does not depend on t.

We derived the same result for MDLM. It is Proposition 6 of Discrete Flow Matching.

Take the mask source once more

A visible token always equals its data token, so the posterior depends only on which data sequences agree with the visible tokens.

Every data sequence that agrees with x gets the same time factor:

$$p_t(x \mid m, x_1) = (1-\kappa_t)^{\#\text{masked}(x)}\ \kappa_t^{\#\text{visible}(x)} \prod_{i\ \text{visible}} \delta_{x_1^i}(x^i)$$

The conditional probability of x given the mask token m and the data sequence x_1 equals one minus κ_t raised to the number of masked positions, times κ_t raised to the number of visible positions, times the product over visible positions of the indicator that the token of x matches x_1.

The time factor cancels in Bayes' rule because it is the same for every candidate x_1. Thus, $p_{1|t}(\cdot \mid x)$ does not depend on t.

With the visible letters fixed, masking time tells us no more about hidden letters.

Now put MDLM’s bound next to our loss

Now put MDLM’s bound next to our loss

Write MDLM’s bound using our current time convention:

$$\int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}\Big[\sum_{i:\, X_t^i = m} -\log p_{1|t}^{\theta}\big(X_1^i \mid X_t\big) \Big]\, dt$$

Now put MDLM’s bound next to our loss

Write MDLM’s bound using our current time convention:

$$\int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\,\mathbb{E}\Big[\sum_{i:\,X_t^i=m} -\log p_{1|t}^{\theta}\big(X_1^i \mid X_t\big)\Big]\,dt$$

The MDLM loss integrates over time the weight $\dot{\kappa}_t$ divided by one minus κ_t, times the expected sum over masked positions of the negative log probability of each true data token.

Now put MDLM’s bound next to our loss

Write MDLM’s bound using our current time convention:

$$\int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\,\mathbb{E}\Big[\sum_{i:\,X_t^i=m} -\log p_{1|t}^{\theta}\big(X_1^i \mid X_t\big)\Big]\,dt$$

The MDLM loss integrates over time the weight $\dot{\kappa}_t$ divided by one minus κ_t, times the expected sum over masked positions of the negative log probability of each true data token.

We remove the weight and include every position in the loss.

Now put MDLM’s bound next to our loss

Write MDLM’s bound using our current time convention:

$$\int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}\Big[\sum_{i:\, X_t^i = m} -\log p_{1|t}^{\theta}\big(X_1^i \mid X_t\big) \Big]\, dt$$

The MDLM loss integrates over time the weight $\dot{\kappa}_t$ divided by one minus κ_t, times the expected sum over masked positions of the negative log probability of each true data token.

We remove the weight and include every position in the loss. At a visible position, the true denoiser is a point mass, so its loss is zero at the optimum.

Now put MDLM’s bound next to our loss

Write MDLM’s bound using our current time convention:

$$\int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\,\mathbb{E}\Big[\sum_{i:\,X_t^i=m} -\log p_{1|t}^{\theta}\big(X_1^i \mid X_t\big)\Big]\,dt$$

The MDLM loss integrates over time the weight $\dot{\kappa}_t$ divided by one minus κ_t, times the expected sum over masked positions of the negative log probability of each true data token.

We remove the weight and include every position in the loss. At a visible position, the true denoiser is a point mass, so its loss is zero at the optimum.

Any positive weight has the same minimizer, so either loss recovers the same denoiser.

Now put MDLM’s bound next to our loss

Write MDLM’s bound using our current time convention:

$$\int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\,\mathbb{E}\Big[\sum_{i:\,X_t^i=m} -\log p_{1|t}^{\theta}\big(X_1^i \mid X_t\big)\Big]\,dt$$

The MDLM loss integrates over time the weight $\dot{\kappa}_t$ divided by one minus κ_t, times the expected sum over masked positions of the negative log probability of each true data token.

We remove the weight and include every position in the loss. At a visible position, the true denoiser is a point mass, so its loss is zero at the optimum.

Any positive weight has the same minimizer, so either loss recovers the same denoiser. Changing the weight changes how much training emphasizes each masking level.

Discrete flow matching sampler

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set $t = 0$.

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set t = 0.
2. Predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position with one network call.

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set $t = 0$.
2. Predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position with one network call.
3. At each position, with probability $\lambda_t = h\dot{\kappa}_t/(1 - \kappa_t)$, redraw the token from the prediction.

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set $t = 0$.
2. Predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position with one network call.
3. At each position, with probability $\lambda_t = h\dot{\kappa}_t/(1 - \kappa_t)$, redraw the token from the prediction.
4. Set $t \leftarrow t + h$, and repeat from step 2 until $t = 1$.

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set $t = 0$.
2. Predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position with one network call.
3. At each position, with probability $\lambda_t = h\dot{\kappa}_t/(1 - \kappa_t)$, redraw the token from the prediction.
4. Set $t \leftarrow t + h$, and repeat from step 2 until $t = 1$.

Near $t = 1$, we keep $\lambda_t \leq 1$ by reducing the step to $h \leq (1 - \kappa_t)/\dot{\kappa}_t$.

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set $t = 0$.
2. Predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position with one network call.
3. At each position, with probability $\lambda_t = h\dot{\kappa}_t/(1 - \kappa_t)$, redraw the token from the prediction.
4. Set $t \leftarrow t + h$, and repeat from step 2 until $t = 1$.

Near $t = 1$, we keep $\lambda_t \leq 1$ by reducing the step to $h \leq (1 - \kappa_t)/\dot{\kappa}_t$. We can also draw every position from the prediction on the last step.

Discrete flow matching sampler

Discrete flow matching sampler

1. Draw x from the source p, and set $t = 0$.
2. Predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position with one network call.
3. At each position, with probability $\lambda_t = h\dot{\kappa}_t/(1 - \kappa_t)$, redraw the token from the prediction.
4. Set $t \leftarrow t + h$, and repeat from step 2 until $t = 1$.

Near $t = 1$, we keep $\lambda_t \leq 1$ by reducing the step to $h \leq (1 - \kappa_t)/\dot{\kappa}_t$. We can also draw every position from the prediction on the last step.

This is Algorithm 1 of the Discrete Flow Matching paper, with the denoiser velocity.

Take a visible position, with the mask source

Take a visible position, with the mask source

At a visible position, the true denoiser puts all of its probability on the visible token.

Take a visible position, with the mask source

At a visible position, the true denoiser puts all of its probability on the visible token.

So the bracket vanishes there: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\delta_{x^i}(\cdot) - \delta_{x^i}(\cdot)\right] = 0$$

Take a visible position, with the mask source

At a visible position, the true denoiser puts all of its probability on the visible token.

So the bracket vanishes there: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\delta_{x^i}(\cdot) - \delta_{x^i}(\cdot)\right] = 0$$

At a visible position, the marginal velocity equals the schedule factor times the point mass on the visible token minus the same point mass, which is zero.

Take a visible position, with the mask source

At a visible position, the true denoiser puts all of its probability on the visible token.

So the bracket vanishes there: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[\delta_{x^i}(\cdot) - \delta_{x^i}(\cdot)\right] = 0$$

At a visible position, the marginal velocity equals the schedule factor times the point mass on the visible token minus the same point mass, which is zero.

We reveal a masked position with probability λ_t by drawing a token from the denoiser.

Take a visible position, with the mask source

At a visible position, the true denoiser puts all of its probability on the visible token.

So the bracket vanishes there: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t}\left[\delta_{x^i}(\cdot) - \delta_{x^i}(\cdot)\right] = 0$$

At a visible position, the marginal velocity equals the schedule factor times the point mass on the visible token minus the same point mass, which is zero.

We reveal a masked position with probability λ_t by drawing a token from the denoiser.

This is MDLM's carry-over unmasking, with the first-order step λ_t in place of $(\kappa_{t+h} - \kappa_t)/(1 - \kappa_t)$.

Now switch to the uniform source

Now switch to the uniform source

A visible token may be a random source token, so the denoiser spreads its probability over all four bases.

Now switch to the uniform source

A visible token may be a random source token, so the denoiser spreads its probability over all four bases.

For our current token G, the denoiser probability is only 0.2.

Now switch to the uniform source

A visible token may be a random source token, so the denoiser spreads its probability over all four bases.

For our current token G, the denoiser probability is only 0.2. We therefore replace G with probability 0.16 in one step.

Now switch to the uniform source

A visible token may be a random source token, so the denoiser spreads its probability over all four bases.

For our current token G, the denoiser probability is only 0.2. We therefore replace G with probability 0.16 in one step.

We can revise an earlier token once the surrounding sequence makes a different token more likely.

Now switch to the uniform source

A visible token may be a random source token, so the denoiser spreads its probability over all four bases.

For our current token G, the denoiser probability is only 0.2. We therefore replace G with probability 0.16 in one step.

We can revise an earlier token once the surrounding sequence makes a different token more likely.

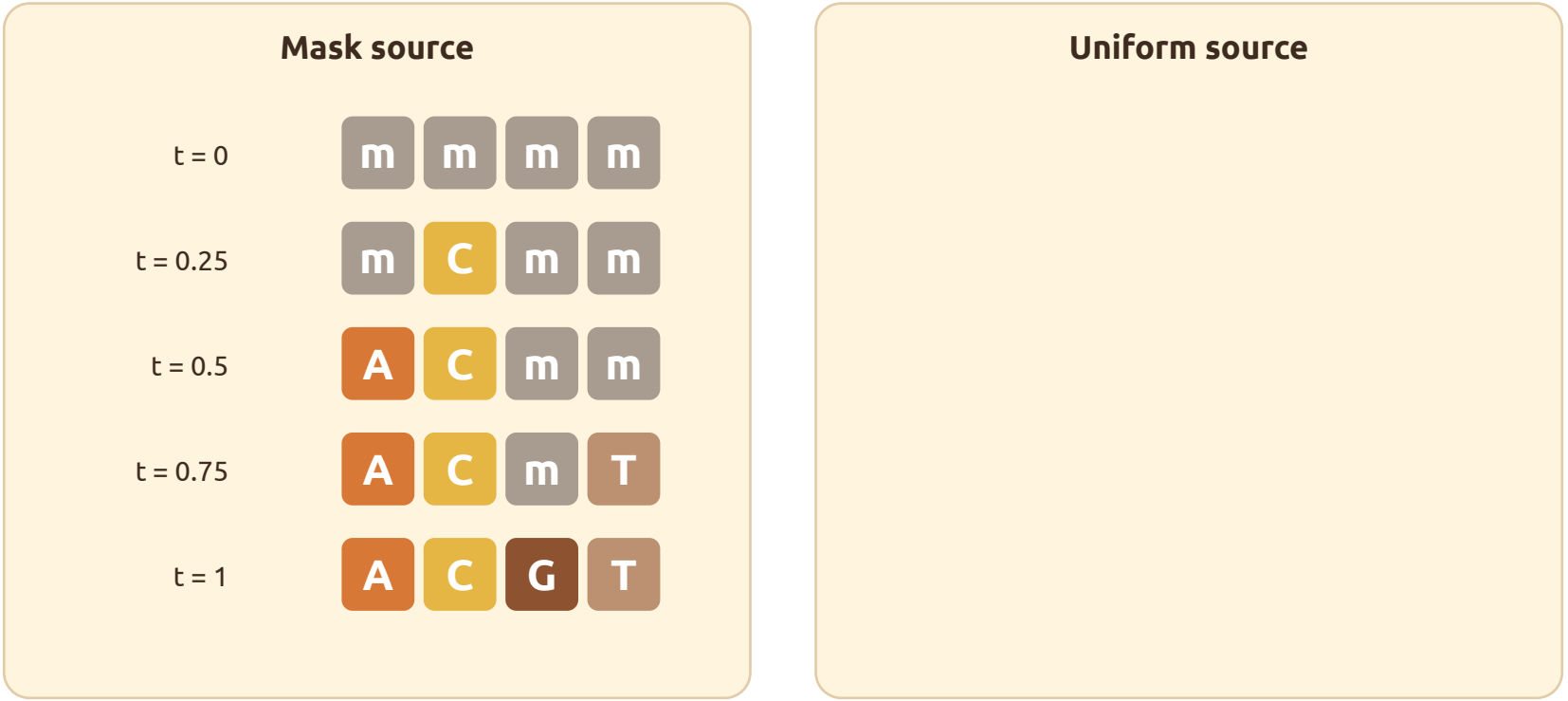
We can revise tokens with a uniform source. With a mask source, revealed tokens stay fixed.

Let’s generate four bases from each source

Mask source

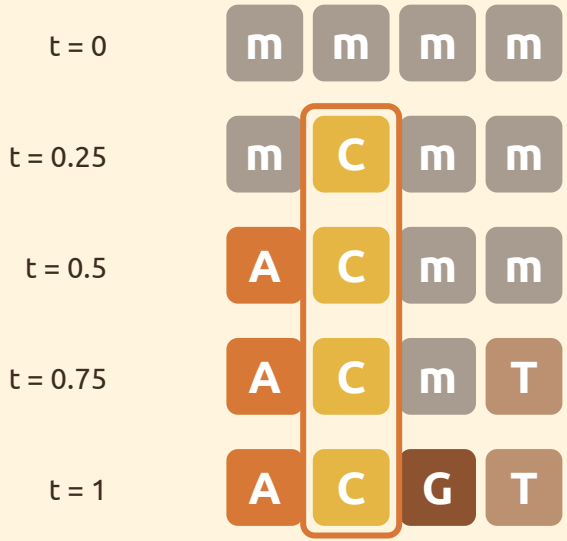
Uniform source

From the mask source, each step reveals some masked positions



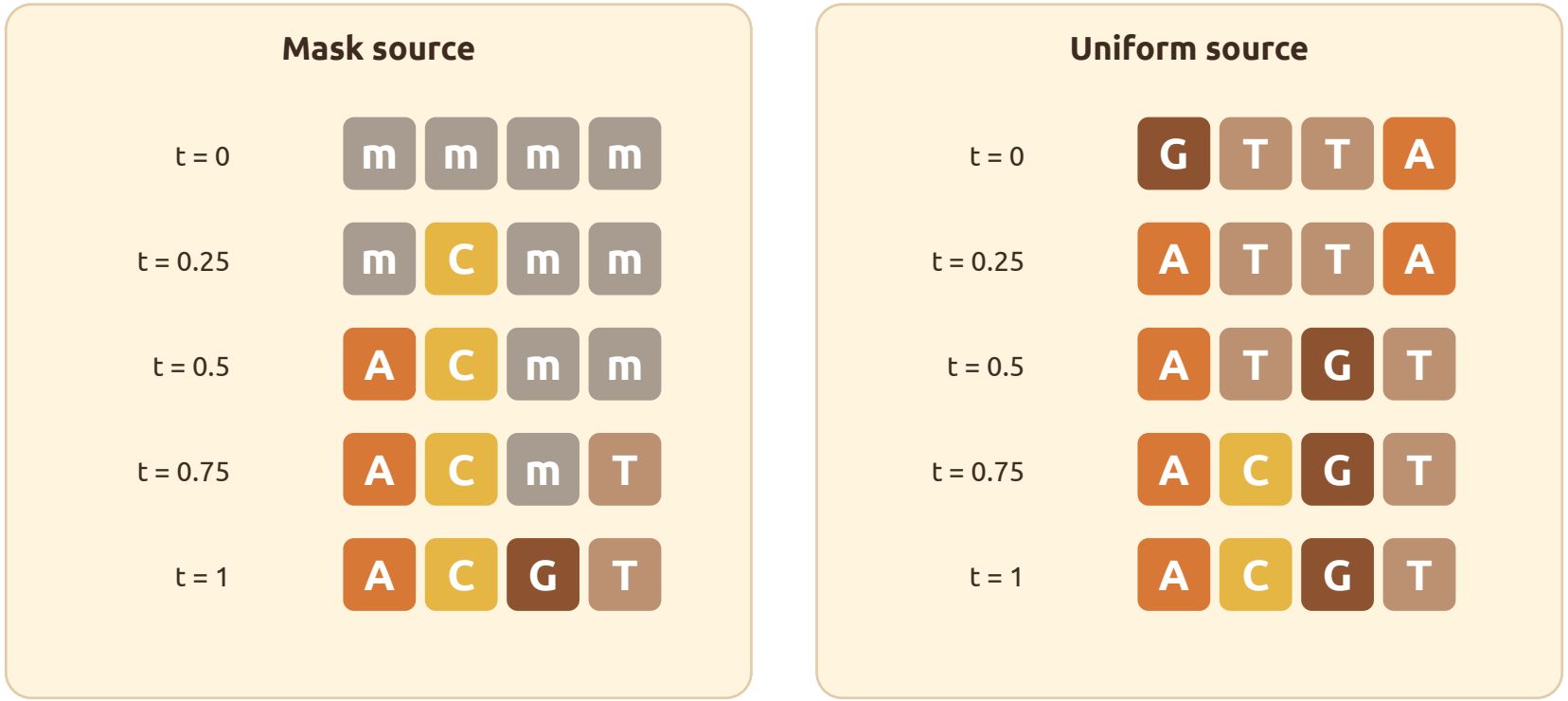
A revealed base stays fixed until the end

Mask source

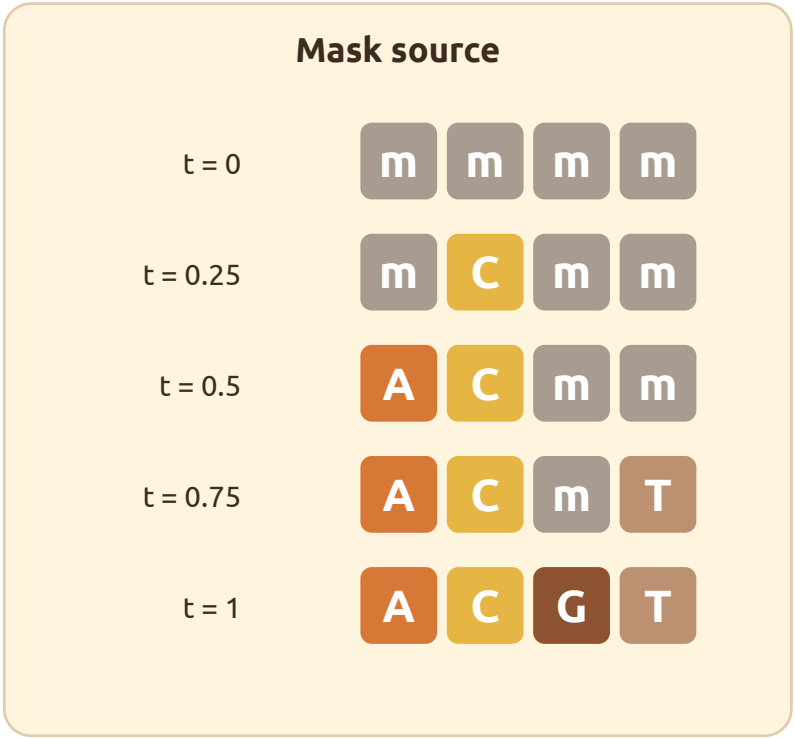


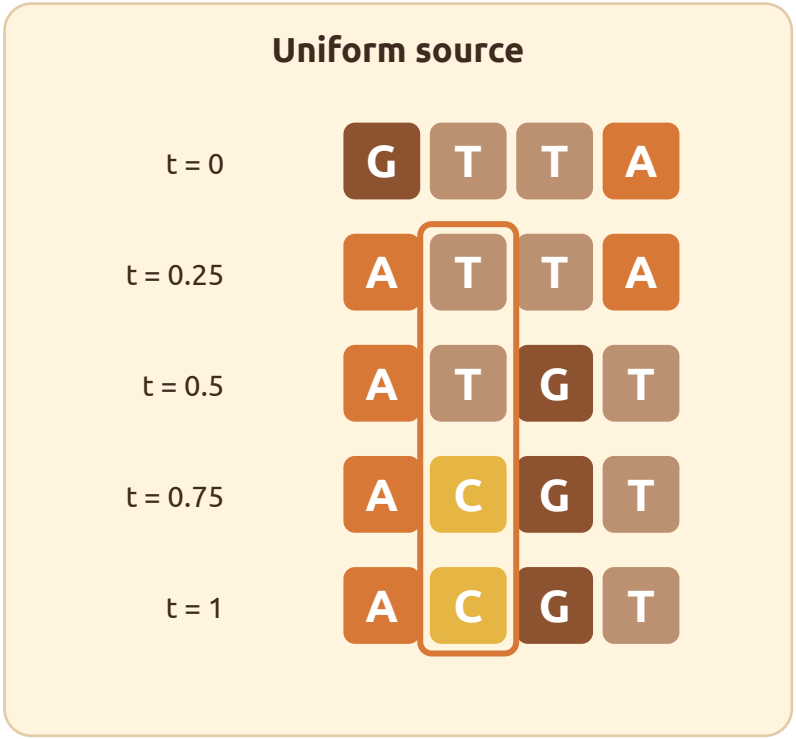
Uniform source

From a uniform source, any base can be redrawn

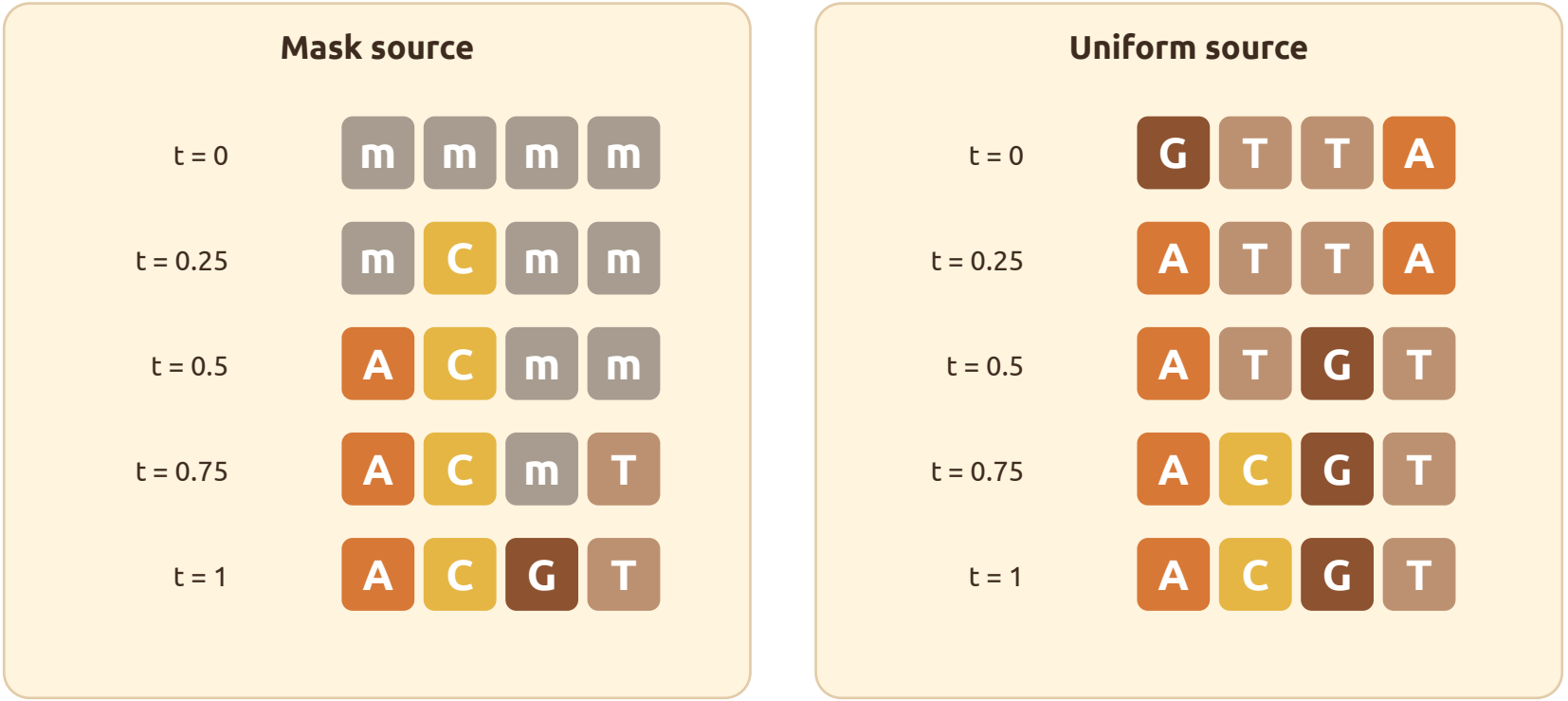


Here a T placed early is revised to the data token C





With masking, we reveal each position once. With a uniform source, we can revise it.



Let’s write discrete flow matching for DNA

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

We specify the vocabulary size and schedule

- The schedule κ_t and its derivative $\dot{\kappa}_t$ are arguments, with $\kappa_t = t$ as the default.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

We draw a random source base at every position

- The schedule κ_t and its derivative $\dot{\kappa}_t$ are arguments, with $\kappa_t = t$ as the default.
- We use a uniform source over the four bases. A mask source would return the mask token.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)    # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)         # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)             # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

We sample the mixture path with one coin flip per position

- Heads, with probability κ_t, keeps the data token. Tails keeps the source token.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

The loss is cross-entropy on every position

- Heads, with probability κ_t, keeps the data token. Tails keeps the source token.
- We score the true data token at every position, with no time weight.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

We first evaluate the denoiser

- With one network call, we predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)                    # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

The redraw probability is h times $\dot{\kappa}_t/(1 - \kappa_t)$

- With one network call, we predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position.
- We clamp the denominator to keep the rate finite near t = 1. When the computed redraw probability exceeds one, we redraw every position.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

We sample new tokens at the selected positions

- With one network call, we predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position.
- We clamp the denominator to keep the rate finite near t = 1. When the computed redraw probability exceeds one, we redraw every position.
- At each position, we either keep the current token or redraw it from $p^{\theta}_{1|t}$.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

The implementation of discrete flow matching

- With one network call, we predict $p^{\theta}_{1|t}(\cdot \mid x)$ at every position.
- We clamp the denominator to keep the rate finite near t = 1. When the computed redraw probability exceeds one, we redraw every position.
- At each position, we either keep the current token or redraw it from $p^{\theta}_{1|t}$.

```python
class DiscreteFlowMatching:
    def __init__(self, V, kappa=lambda t: t,
                 kappa_dot=lambda t: torch.ones_like(t)):
        self.V, self.k, self.kd = V, kappa, kappa_dot

    def source(self, shape):
        return torch.randint(0, self.V, shape)   # uniform

    def corrupt(self, x1, t):
        x0 = self.source(x1.shape)
        heads = torch.rand(x1.shape) < self.k(t)[:, None]
        return torch.where(heads, x1, x0)        # mixture

    def loss(self, logits, x1):
        return F.cross_entropy(logits.transpose(1, 2), x1)

    @torch.no_grad()
    def step(self, model, xt, t, h):
        p1 = model(xt, t).softmax(-1)            # denoiser
        rate = self.kd(t) / (1 - self.k(t)).clamp(min=1e-4)
        redraw = torch.rand(xt.shape) < (h * rate)[:, None]
        x1 = Categorical(probs=p1).sample()
        return torch.where(redraw, x1, xt)
```

Training and generation are two short loops

```python
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

We draw one training time per sequence

- Each sequence in the batch gets its own time and its own noisy version.

```python
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

We pass the noisy sequence and the time to the network

- Each sequence in the batch gets its own time and its own noisy version.
- With the mask source, we could drop t from the network input.

```python
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

Generation starts from the source

- It takes sixty-four steps from t = 0 to t = 1.

```python
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

Each step moves the sequence forward by s – t

- It takes sixty-four steps from t = 0 to t = 1.
- We did not fix a step size during training, so we can choose it at sampling time.

```python
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

The same loops work for every path in this lecture

- It takes sixty-four steps from t = 0 to t = 1.
- We did not fix a step size during training, so we can choose it at sampling time.

```
def train_step(model, proc, x1, opt):
    t = torch.rand(x1.shape[0])
    xt = proc.corrupt(x1, t)
    loss = proc.loss(model(xt, t), x1)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()

@torch.no_grad()
def generate(model, proc, B, L, steps=64):
    x = proc.source((B, L))
    ts = torch.linspace(0.0, 1.0, steps + 1)
    for t, s in zip(ts[:-1], ts[1:]):
        x = proc.step(model, x, t.expand(B), s - t)
    return x
```

Discrete Flow Matching: revealing one G

Toy masked DNA position at time 0.6, with the linear mixture schedule. The denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Take a first-order step of length 0.1.

Discrete Flow Matching: revealing one G

Toy masked DNA position at time 0.6, with the linear mixture schedule. The denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Take a first-order step of length 0.1.

$$u(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T} \mid m) = \frac{(0.1, 0.2, 0.6, 0.1)}{1 - 0.6} = (0.25, 0.5, 1.5, 0.25)$$

Discrete Flow Matching: revealing one G

Toy masked DNA position at time 0.6, with the linear mixture schedule. The denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Take a first-order step of length 0.1.

$$u(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T} \mid m) = \frac{(0.1, 0.2, 0.6, 0.1)}{1 - 0.6} = (0.25, 0.5, 1.5, 0.25)$$

Divide the denoiser probabilities by the remaining time. The total reveal rate is 2.5.

Discrete Flow Matching: revealing one G

Toy masked DNA position at time 0.6, with the linear mixture schedule. The denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Take a first-order step of length 0.1.

$$u(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T} \mid m) = \frac{(0.1, 0.2, 0.6, 0.1)}{1 - 0.6} = (0.25, 0.5, 1.5, 0.25)$$

Divide the denoiser probabilities by the remaining time. The total reveal rate is 2.5.

$$p_{\mathrm{step}}(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}, m) = (0.025, 0.05, 0.15, 0.025, 0.75)$$

Discrete Flow Matching: revealing one G

Toy masked DNA position at time 0.6, with the linear mixture schedule. The denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Take a first-order step of length 0.1.

$$u(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T} \mid m) = \frac{(0.1, 0.2, 0.6, 0.1)}{1 - 0.6} = (0.25, 0.5, 1.5, 0.25)$$

Divide the denoiser probabilities by the remaining time. The total reveal rate is 2.5.

$$p_{\mathrm{step}}(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}, m) = (0.025, 0.05, 0.15, 0.025, 0.75)$$

G appears with probability 0.15 in this step. The position stays masked with probability 0.75.

Discrete Flow Matching: revealing one G

Toy masked DNA position at time 0.6, with the linear mixture schedule. The denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Take a first-order step of length 0.1.

$$u(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T} \mid m) = \frac{(0.1, 0.2, 0.6, 0.1)}{1 - 0.6} = (0.25, 0.5, 1.5, 0.25)$$

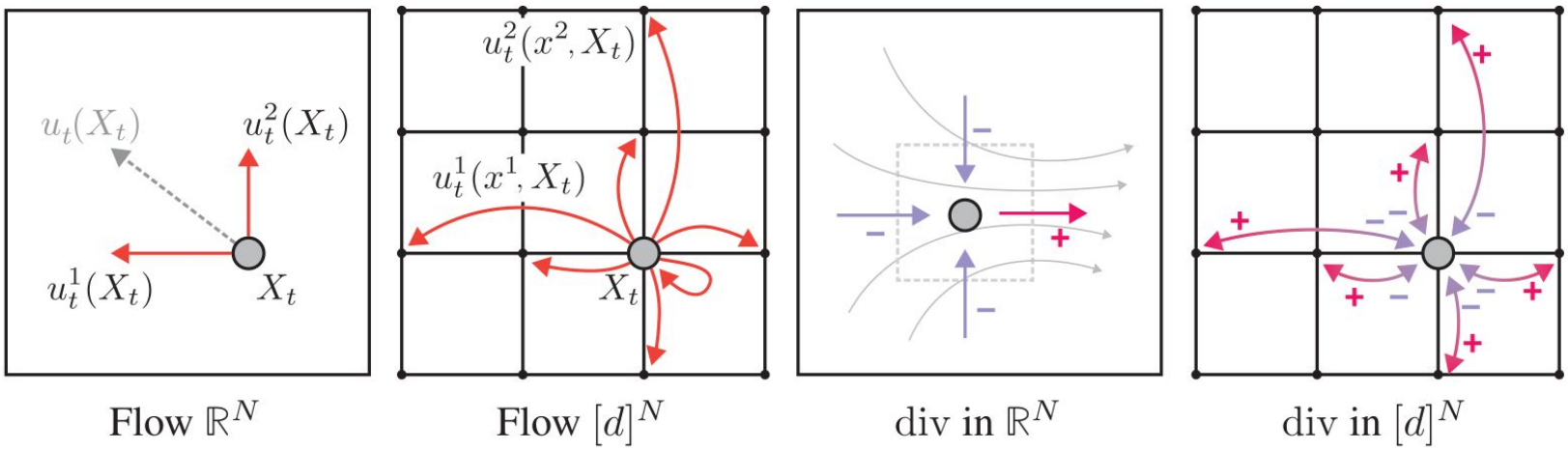
Divide the denoiser probabilities by the remaining time. The total reveal rate is 2.5.

$$p_{\mathrm{step}}(\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}, m) = (0.025, 0.05, 0.15, 0.025, 0.75)$$

G appears with probability 0.15 in this step. The position stays masked with probability 0.75.

A rate becomes a sampling probability only after we choose how far to advance in time.

Single-token jumps generate the probability path



We generate the path through single-token jumps, using the factorized velocity.

Gat,..,Lipman. *NeurIPS* (2024)

Now look at the generative perplexity in Table 2 of the paper

Table 2: Generative perplexity on unconditional text generation compared to prior work. All models are sampled without the use of temperature or corrector steps. Double precision sampling results are reported in Table 5.

METHOD	NFE	LLAMA-2↓	LLAMA-3↓	GPT2↓	ENTROPY
Data	-	7.0	9.4	14.7	7.7
Autoregressive	1024	31.4	54.8	45.3	7.1
Savinov et al. (2021)	200	29.5	45.1	34.7	5.2
Austin et al. (2021a)	1000	697.6	768.8	837.8	7.6
Han et al. (2022)	>10000	73.3	203.1	99.2	4.8
Lou et al. (2023)	256/512/1024	38.6/33.7/27.2	69.2/58.6/43.9	64.3/53.4/40.5	7.8/7.7/7.6
Campbell et al. (2024)	256/512/1024	38.5/33.5/28.7	69.0/56.5/46.5	65.2/53.3/43.0	7.8/7.7/7.6
FM (equation 9)	256/512/1024	34.2/30.0/22.5	58.5/48.8/33.8	54.2/43.5/29.3	7.7/7.6/7.2
FM (equation 10)	256/512/1024	30.0/27.5/22.3	48.2/43.5/31.9	47.7/41.8/28.1	7.6/7.5/7.1

Gat,..,Lipman. *NeurIPS* (2024)

The linear schedule $\kappa_t = t$ from Discrete Flow Models scores 28.7 in Llama-2 perplexity at 1024 steps

Table 2: Generative perplexity on unconditional text generation compared to prior work. All models are sampled without the use of temperature or corrector steps. Double precision sampling results are reported in Table 5.

METHOD	NFE	LLAMA-2↓	LLAMA-3↓	GPT2↓	ENTROPY
Data	-	7.0	9.4	14.7	7.7
Autoregressive	1024	31.4	54.8	45.3	7.1
Savinov et al. (2021)	200	29.5	45.1	34.7	5.2
Austin et al. (2021a)	1000	697.6	768.8	837.8	7.6
Han et al. (2022)	>10000	73.3	203.1	99.2	4.8
Lou et al. (2023)	256/512/1024	38.6/33.7/27.2	69.2/58.6/43.9	64.3/53.4/40.5	7.8/7.7/7.6
Campbell et al. (2024)	256/512/1024	38.5/33.5/28.7	69.0/56.5/46.5	65.2/53.3/43.0	7.8/7.7/7.6
FM (equation 9)	256/512/1024	34.2/30.0/22.5	58.5/48.8/33.8	54.2/43.5/29.3	7.7/7.6/7.2
FM (equation 10)	256/512/1024	30.0/27.5/22.3	48.2/43.5/31.9	47.7/41.8/28.1	7.6/7.5/7.1

Gat,..,Lipman. *NeurIPS* (2024)

With the same mixture path and $\kappa_t = t^2$, perplexity improves to 22.5

Table 2: Generative perplexity on unconditional text generation compared to prior work. All models are sampled without the use of temperature or corrector steps. Double precision sampling results are reported in Table 5.

METHOD	NFE	LLAMA-2↓	LLAMA-3↓	GPT2↓	ENTROPY
Data	-	7.0	9.4	14.7	7.7
Autoregressive	1024	31.4	54.8	45.3	7.1
Savinov et al. (2021)	200	29.5	45.1	34.7	5.2
Austin et al. (2021a)	1000	697.6	768.8	837.8	7.6
Han et al. (2022)	>10000	73.3	203.1	99.2	4.8
Lou et al. (2023)	256/512/1024	38.6/33.7/27.2	69.2/58.6/43.9	64.3/53.4/40.5	7.8/7.7/7.6
Campbell et al. (2024)	256/512/1024	38.5/33.5/28.7	69.0/56.5/46.5	65.2/53.3/43.0	7.8/7.7/7.6
FM (equation 9)	256/512/1024	34.2/30.0/22.5	58.5/48.8/33.8	54.2/43.5/29.3	7.7/7.6/7.2
FM (equation 10)	256/512/1024	30.0/27.5/22.3	48.2/43.5/31.9	47.7/41.8/28.1	7.6/7.5/7.1

Gat,..,Lipman. *NeurIPS* (2024)

The t^2 schedule is best at 256, 512, and 1024 steps, with no change to the form of the training loss

Table 2: Generative perplexity on unconditional text generation compared to prior work. All models are sampled without the use of temperature or corrector steps. Double precision sampling results are reported in Table 5.

METHOD	NFE	LLAMA-2↓	LLAMA-3↓	GPT2↓	ENTROPY
Data	-	7.0	9.4	14.7	7.7
Autoregressive	1024	31.4	54.8	45.3	7.1
Savinov et al. (2021)	200	29.5	45.1	34.7	5.2
Austin et al. (2021a)	1000	697.6	768.8	837.8	7.6
Han et al. (2022)	>10000	73.3	203.1	99.2	4.8
Lou et al. (2023)	256/512/1024	38.6/33.7/27.2	69.2/58.6/43.9	64.3/53.4/40.5	7.8/7.7/7.6
Campbell et al. (2024)	256/512/1024	38.5/33.5/28.7	69.0/56.5/46.5	65.2/53.3/43.0	7.8/7.7/7.6
FM (equation 9)	256/512/1024	34.2/30.0/22.5	58.5/48.8/33.8	54.2/43.5/29.3	7.7/7.6/7.2
FM (equation 10)	256/512/1024	30.0/27.5/22.3	48.2/43.5/31.9	47.7/41.8/28.1	7.6/7.5/7.1

Gat,..,Lipman. *NeurIPS* (2024)

With double precision in their appendix, the discrete diffusion and flow models have perplexity near 50, compared with 31.4 for the autoregressive model

Table 2: Generative perplexity on unconditional text generation compared to prior work. All models are sampled without the use of temperature or corrector steps. Double precision sampling results are reported in Table 5.

METHOD	NFE	LLAMA-2↓	LLAMA-3↓	GPT2↓	ENTROPY
Data	-	7.0	9.4	14.7	7.7
Autoregressive	1024	31.4	54.8	45.3	7.1
Savinov et al. (2021)	200	29.5	45.1	34.7	5.2
Austin et al. (2021a)	1000	697.6	768.8	837.8	7.6
Han et al. (2022)	>10000	73.3	203.1	99.2	4.8
Lou et al. (2023)	256/512/1024	38.6/33.7/27.2	69.2/58.6/43.9	64.3/53.4/40.5	7.8/7.7/7.6
Campbell et al. (2024)	256/512/1024	38.5/33.5/28.7	69.0/56.5/46.5	65.2/53.3/43.0	7.8/7.7/7.6
FM (equation 9)	256/512/1024	34.2/30.0/22.5	58.5/48.8/33.8	54.2/43.5/29.3	7.7/7.6/7.2
FM (equation 10)	256/512/1024	30.0/27.5/22.3	48.2/43.5/31.9	47.7/41.8/28.1	7.6/7.5/7.1

Gat,..,Lipman. *NeurIPS* (2024)

At 1.7B parameters, they train the same recipe on code

Table 4: Execution based code generation evaluation.

METHOD	DATA	HUMANEVAL↑			MBPP (1-SHOT)↑		
		Pass@1	Pass@10	Pass@25	Pass@1	Pass@10	Pass@25
Autoregressive	Text	1.2	3.1	4.8	0.2	1.7	3.3
	Code	14.3	21.3	27.8	17.0	34.3	44.1
FM	Text	1.2	2.6	4.0	0.4	1.1	3.6
	Code	6.7	13.4	18.0	6.7	20.6	26.5
FM (Oracle length)	Code	11.6	18.3	20.6	13.1	28.4	34.2

Gat,..,Lipman. *NeurIPS* (2024)

Trained on code, it solves 6.7% of HumanEval problems on the first try and 13.4% within ten

Table 4: Execution based code generation evaluation.

METHOD	DATA	HUMANEVAL↑			MBPP (1-SHOT)↑		
		Pass@1	Pass@10	Pass@25	Pass@1	Pass@10	Pass@25
Autoregressive	Text	1.2	3.1	4.8	0.2	1.7	3.3
	Code	14.3	21.3	27.8	17.0	34.3	44.1
FM	Text	1.2	2.6	4.0	0.4	1.1	3.6
	Code	6.7	13.4	18.0	6.7	20.6	26.5
FM (Oracle length)	Code	11.6	18.3	20.6	13.1	28.4	34.2

Gat,..,Lipman. *NeurIPS* (2024)

On MBPP, it reaches 6.7% on the first try and 20.6% within ten

Table 4: Execution based code generation evaluation.

METHOD	DATA	HUMANEVAL↑			MBPP (1-SHOT)↑		
		Pass@1	Pass@10	Pass@25	Pass@1	Pass@10	Pass@25
Autoregressive	Text	1.2	3.1	4.8	0.2	1.7	3.3
	Code	14.3	21.3	27.8	17.0	34.3	44.1
FM	Text	1.2	2.6	4.0	0.4	1.1	3.6
	Code	6.7	13.4	18.0	6.7	20.6	26.5
FM (Oracle length)	Code	11.6	18.3	20.6	13.1	28.4	34.2

Gat,..,Lipman. *NeurIPS* (2024)

An autoregressive model of the same size still leads, at 14.3% on HumanEval and 17.0% on MBPP, both on the first try

Table 4: Execution based code generation evaluation.

METHOD	DATA	HUMANEVAL↑			MBPP (1-SHOT)↑		
		Pass@1	Pass@10	Pass@25	Pass@1	Pass@10	Pass@25
Autoregressive	Text	1.2	3.1	4.8	0.2	1.7	3.3
	Code	14.3	21.3	27.8	17.0	34.3	44.1
FM	Text	1.2	2.6	4.0	0.4	1.1	3.6
	Code	6.7	13.4	18.0	6.7	20.6	26.5
FM (Oracle length)	Code	11.6	18.3	20.6	13.1	28.4	34.2

Gat,..,Lipman. *NeurIPS* (2024)

We can generate working code by filling in masked tokens

Table 4: Execution based code generation evaluation.

METHOD	DATA	HUMANEVAL↑			MBPP (1-SHOT)↑		
		Pass@1	Pass@10	Pass@25	Pass@1	Pass@10	Pass@25
Autoregressive	Text	1.2	3.1	4.8	0.2	1.7	3.3
	Code	14.3	21.3	27.8	17.0	34.3	44.1
FM	Text	1.2	2.6	4.0	0.4	1.1	3.6
	Code	6.7	13.4	18.0	6.7	20.6	26.5
FM (Oracle length)	Code	11.6	18.3	20.6	13.1	28.4	34.2

Gat,..,Lipman. *NeurIPS* (2024)

Now look again at the master equation

Now look again at the master equation

Our conditional velocity was one solution of the master equation. It is not the only one.

Now look again at the master equation

Our conditional velocity was one solution of the master equation. It is not the only one.

$$\sum_x v_t(y,x)\, p_t(x) = 0 \quad \text{for every } y \quad \Longrightarrow \quad u_t + v_t \text{ also generates } p_t$$

Now look again at the master equation

Our conditional velocity was one solution of the master equation. It is not the only one.

$$\sum_x v_t(y,x)\,p_t(x) = 0 \quad \text{for every } y \quad \Longrightarrow \quad u_t + v_t \text{ also generates } p_t$$

If a velocity v_t has zero net flux into every state y, meaning the sum over x of $v_t(y, x)$ times $p_t(x)$ is zero, then adding it to a velocity u_t that generates p_t gives another velocity that generates the same path.

Now look again at the master equation

Our conditional velocity was one solution of the master equation. It is not the only one.

$$\sum_x v_t(y,x)\,p_t(x) = 0 \quad \text{for every } y \quad \Longrightarrow \quad u_t + v_t \text{ also generates } p_t$$

If a velocity v_t has zero net flux into every state y, meaning the sum over x of $v_t(y, x)$ times $p_t(x)$ is zero, then adding it to a velocity u_t that generates p_t gives another velocity that generates the same path.

The master equation is linear in the velocity, so the extra term adds nothing to the rate of change, as long as the sum has nonnegative rates off the diagonal.

Now look again at the master equation

Our conditional velocity was one solution of the master equation. It is not the only one.

$$\sum_x v_t(y,x)\,p_t(x) = 0 \quad \text{for every } y \quad \Longrightarrow \quad u_t + v_t \text{ also generates } p_t$$

If a velocity v_t has zero net flux into every state y, meaning the sum over x of $v_t(y, x)$ times $p_t(x)$ is zero, then adding it to a velocity u_t that generates p_t gives another velocity that generates the same path.

The master equation is linear in the velocity, so the extra term adds nothing to the rate of change, as long as the sum has nonnegative rates off the diagonal.

We can add this term to revise individual samples without changing the marginals.

Now look again at the master equation

Our conditional velocity was one solution of the master equation. It is not the only one.

$$\sum_x v_t(y,x)\,p_t(x) = 0 \quad \text{for every } y \quad \Longrightarrow \quad u_t + v_t \text{ also generates } p_t$$

If a velocity v_t has zero net flux into every state y, meaning the sum over x of $v_t(y, x)$ times $p_t(x)$ is zero, then adding it to a velocity u_t that generates p_t gives another velocity that generates the same path.

The master equation is linear in the velocity, so the extra term adds nothing to the rate of change, as long as the sum has nonnegative rates off the diagonal.

Extra jumps can rearrange individual samples while arrivals and departures cancel at every state.

Detailed balance for the corrector

Detailed balance for the corrector

As in discrete diffusion, detailed balance requires equal flux in both directions:

$$\boxed{v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y}$$

Detailed balance for the corrector

As in discrete diffusion, detailed balance requires equal flux in both directions:

$$\boxed{v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y}$$

The flux from x to y, $v_t(y, x)$ times $p_t(x)$, equals the flux from y to x, $v_t(x, y)$ times $p_t(y)$, for every pair of different states.

Detailed balance for the corrector

As in discrete diffusion, detailed balance requires equal flux in both directions:

$$\boxed{v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y}$$

The flux from x to y, $v_t(y, x)$ times $p_t(x)$, equals the flux from y to x, $v_t(x, y)$ times $p_t(y)$, for every pair of different states.

Then the net flux into any state y vanishes:

$$\sum_x v_t(y, x)\, p_t(x) = \sum_x v_t(x, y)\, p_t(y) = p_t(y) \sum_x v_t(x, y) = 0$$

Detailed balance for the corrector

As in discrete diffusion, detailed balance requires equal flux in both directions:

$$\boxed{v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y}$$

The flux from x to y, $v_t(y, x)$ times $p_t(x)$, equals the flux from y to x, $v_t(x, y)$ times $p_t(y)$, for every pair of different states.

Then the net flux into any state y vanishes:

$$\sum_x v_t(y, x)\, p_t(x) = \sum_x v_t(x, y)\, p_t(y) = p_t(y) \sum_x v_t(x, y) = 0$$

The net flux into y is $p_t(y)$ times the sum of the rates $v_t(x, y)$ leaving y, including the diagonal. These rates sum to zero.

Detailed balance for the corrector

As in discrete diffusion, detailed balance requires equal flux in both directions:

$$\boxed{v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y}$$

The flux from x to y, $v_t(y, x)$ times $p_t(x)$, equals the flux from y to x, $v_t(x, y)$ times $p_t(y)$, for every pair of different states.

Then the net flux into any state y vanishes:

$$\sum_x v_t(y, x)\, p_t(x) = \sum_x v_t(x, y)\, p_t(y) = p_t(y) \sum_x v_t(x, y) = 0$$

Detailed balance makes p_t stationary under the extra term.

Detailed balance for the corrector

As in discrete diffusion, detailed balance requires equal flux in both directions:

$$\boxed{v_t(y, x)\, p_t(x) = v_t(x, y)\, p_t(y) \qquad \text{for all } x \neq y}$$

The flux from x to y, $v_t(y, x)$ times $p_t(x)$, equals the flux from y to x, $v_t(x, y)$ times $p_t(y)$, for every pair of different states.

Then the net flux into any state y vanishes:

$$\sum_x v_t(y, x)\, p_t(x) = \sum_x v_t(x, y)\, p_t(y) = p_t(y) \sum_x v_t(x, y) = 0$$

Detailed balance makes p_t stationary under the extra term. The original velocity u_t still changes p_t along the prescribed path.

In Discrete Flow Matching, we also add a corrector

In Discrete Flow Matching, we also add a corrector

Run the same path backward in time, with the source posterior $p_{0|t}$:

$$\check{u}_t^i(y^i, x) = \frac{\dot{\kappa}_t}{\kappa_t} \left[\delta_{x^i}(y^i) - p_{0|t}(y^i \mid x) \right]$$

In Discrete Flow Matching, we also add a corrector

Run the same path backward in time, with the source posterior $p_{0|t}$:

$$\check{u}_t^i(y^i, x) = \frac{\dot{\kappa}_t}{\kappa_t} \left[\delta_{x^i}(y^i) - p_{0|t}(y^i \mid x) \right]$$

The backward velocity at position i equals $\dot{\kappa}_t$ divided by κ_t, times the indicator that y^i is the current token x^i, minus the posterior probability $p_{0|t}(y^i \mid x)$ that the source token was y^i.

In Discrete Flow Matching, we also add a corrector

Run the same path backward in time, with the source posterior $p_{0|t}$:

$$\check{u}_t^i(y^i, x) = \frac{\dot{\kappa}_t}{\kappa_t} \left[\delta_{x^i}(y^i) - p_{0|t}(y^i \mid x) \right]$$

The backward velocity at position i equals $\dot{\kappa}_t$ divided by κ_t, times the indicator that y^i is the current token x^i, minus the posterior probability $p_{0|t}(y^i \mid x)$ that the source token was y^i.

It has the same shape as the forward velocity, with the source posterior in place of the denoiser, κ_t in place of $1 - \kappa_t$, and the two terms in the bracket exchanged.

Now combine the forward and backward velocities

Now combine the forward and backward velocities

$$\boxed{\bar{u}_t = \alpha_t \, \hat{u}_t - \beta_t \, \check{u}_t, \qquad \alpha_t - \beta_t = 1}$$

Now combine the forward and backward velocities

$$\boxed{\bar{u}_t = \alpha_t \, \hat{u}_t - \beta_t \, \check{u}_t, \qquad \alpha_t - \beta_t = 1}$$

The corrector velocity $\bar{u}_t$ equals α_t times the forward velocity $\hat{u}_t$ minus β_t times the backward velocity $\check{u}_t$, where α_t minus β_t equals one.

Now combine the forward and backward velocities

$$\boxed{\bar{u}_t = \alpha_t \, \hat{u}_t - \beta_t \, \check{u}_t, \qquad \alpha_t - \beta_t = 1}$$

The corrector velocity $\bar{u}_t$ equals α_t times the forward velocity $\hat{u}_t$ minus β_t times the backward velocity $\check{u}_t$, where α_t minus β_t equals one.

At each step, we combine forward motion weighted by $1 + \beta_t$ with backward motion weighted by β_t.

Now combine the forward and backward velocities

$$\boxed{\bar{u}_t = \alpha_t \, \hat{u}_t - \beta_t \, \check{u}_t, \qquad \alpha_t - \beta_t = 1}$$

The corrector velocity $\bar{u}_t$ equals α_t times the forward velocity $\hat{u}_t$ minus β_t times the backward velocity $\check{u}_t$, where α_t minus β_t equals one.

At each step, we combine forward motion weighted by $1 + \beta_t$ with backward motion weighted by β_t. We therefore add noise and remove it again.

Now combine the forward and backward velocities

$$\boxed{\bar{u}_t = \alpha_t\,\hat{u}_t - \beta_t\,\check{u}_t, \qquad \alpha_t - \beta_t = 1}$$

The corrector velocity $\bar{u}_t$ equals α_t times the forward velocity $\hat{u}_t$ minus β_t times the backward velocity $\check{u}_t$, where α_t minus β_t equals one.

At each step, we combine forward motion weighted by $1 + \beta_t$ with backward motion weighted by β_t. We therefore add noise and remove it again.

The difference $\hat{u}_t - \check{u}_t$ satisfies detailed balance with p_t, so $\bar{u}_t$ still generates the same path.

Now combine the forward and backward velocities

$$\boxed{\bar{u}_t = \alpha_t \, \hat{u}_t - \beta_t \, \check{u}_t, \qquad \alpha_t - \beta_t = 1}$$

The corrector velocity $\bar{u}_t$ equals α_t times the forward velocity $\hat{u}_t$ minus β_t times the backward velocity $\check{u}_t$, where α_t minus β_t equals one.

At each step, we combine forward motion weighted by $1 + \beta_t$ with backward motion weighted by β_t. We therefore add noise and remove it again.

Extra forward and backward motion creates chances to revise tokens while preserving the same probability path.

Write the corrector for the mask source

Write the corrector for the mask source

Take $\kappa_t = t$, $\alpha_t = 1 + \eta t$, and $\beta_t = \eta t$, with the mask source, where $p_{0|t}$ is the mask itself.

Write the corrector for the mask source

Take $\kappa_t = t$, $\alpha_t = 1 + \eta t$, and $\beta_t = \eta t$, with the mask source, where $p_{0|t}$ is the mask itself.

$$\boxed{\bar{u}_t^i(\cdot, x) = \frac{1+\eta t}{1-t}\left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot)\right] + \eta\left[\delta_m(\cdot) - \delta_{x^i}(\cdot)\right]}$$

Write the corrector for the mask source

Take $\kappa_t = t$, $\alpha_t = 1 + \eta t$, and $\beta_t = \eta t$, with the mask source, where $p_{0|t}$ is the mask itself.

$$\boxed{\bar{u}_t^i(\cdot, x) = \frac{1 + \eta t}{1 - t}\left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot)\right] + \eta\left[\delta_m(\cdot) - \delta_{x^i}(\cdot)\right]}$$

The corrector velocity at position i equals one plus ηt over one minus t, times the denoiser distribution minus the point mass on the current token, plus η times the point mass on the mask minus the point mass on the current token.

Write the corrector for the mask source

Take $\kappa_t = t$, $\alpha_t = 1 + \eta t$, and $\beta_t = \eta t$, with the mask source, where $p_{0|t}$ is the mask itself.

$$\bar{u}_t^i(\cdot, x) = \frac{1 + \eta t}{1 - t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right] + \eta \left[\delta_m(\cdot) - \delta_{x^i}(\cdot) \right]$$

The corrector velocity at position i equals one plus ηt over one minus t, times the denoiser distribution minus the point mass on the current token, plus η times the point mass on the mask minus the point mass on the current token.

A mask unmasks at rate (1 + ηt)/(1 – t). A visible token remasks at rate η.

Write the corrector for the mask source

Take $\kappa_t = t$, $\alpha_t = 1 + \eta t$, and $\beta_t = \eta t$, with the mask source, where $p_{0|t}$ is the mask itself.

$$\boxed{\bar{u}_t^i(\cdot, x) = \frac{1 + \eta t}{1 - t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i}(\cdot) \right] + \eta \left[\delta_m(\cdot) - \delta_{x^i}(\cdot) \right]}$$

The corrector velocity at position i equals one plus ηt over one minus t, times the denoiser distribution minus the point mass on the current token, plus η times the point mass on the mask minus the point mass on the current token.

A mask unmasks at rate (1 + ηt)/(1 – t). A visible token remasks at rate η.

This is the stochastic sampler of Discrete Flow Models, where η is the amount of stochasticity.

Let’s count the jumps at three noise levels

Let’s count the jumps at three noise levels

At $t = 0.4$ and $\eta = 2$, a masked position unmasks at rate 3.0, while a visible position remasks at rate 2.

Let’s count the jumps at three noise levels

At t = 0.4 and η = 2, a masked position unmasks at rate 3.0, while a visible position remasks at rate 2.

Over the whole path, each position jumps on average

$$\eta = 0,\ 2,\ 10: \qquad \mathbb{E}[\text{jumps per position}] = 1 + \eta = 1,\ 3,\ 11$$

Let’s count the jumps at three noise levels

At t = 0.4 and η = 2, a masked position unmasks at rate 3.0, while a visible position remasks at rate 2.

Over the whole path, each position jumps on average

$$\eta = 0,\ 2,\ 10: \qquad \mathbb{E}[\text{jumps per position}] = 1 + \eta = 1,\ 3,\ 11$$

The expected number of jumps per position equals one plus η, which is 1, 3, and 11 for η equal to 0, 2, and 10.

Let’s count the jumps at three noise levels

At t = 0.4 and η = 2, a masked position unmasks at rate 3.0, while a visible position remasks at rate 2.

Over the whole path, each position jumps on average

$$\eta = 0,\ 2,\ 10: \qquad \mathbb{E}[\text{jumps per position}] = 1 + \eta = 1,\ 3,\ 11$$

Across 200,000 simulated paths at each level, the masked fraction at t = 0.5 is 0.50 in all three cases. The final bases match the data distribution.

Let’s count the jumps at three noise levels

At t = 0.4 and η = 2, a masked position unmasks at rate 3.0, while a visible position remasks at rate 2.

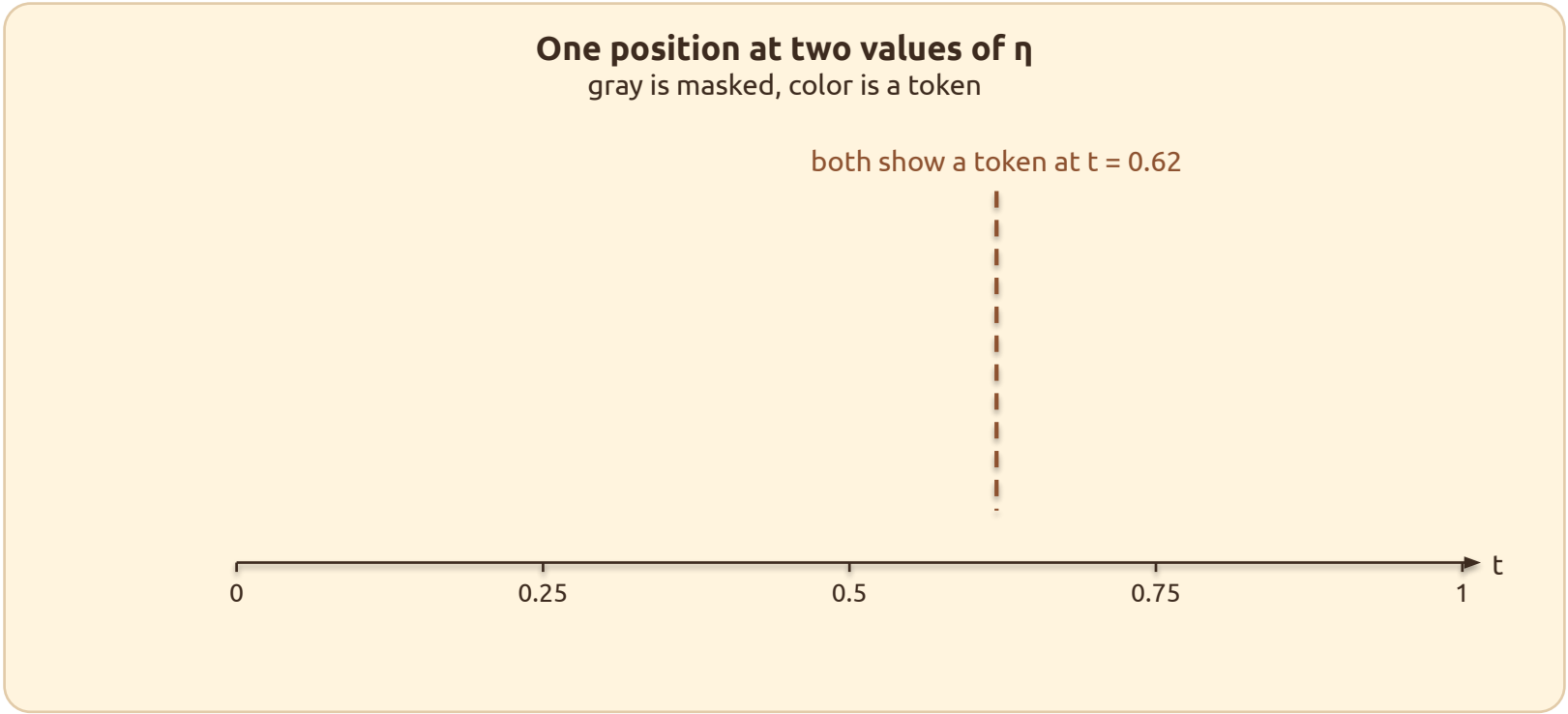
Over the whole path, each position jumps on average

$$\eta = 0,\ 2,\ 10: \qquad \mathbb{E}[\text{jumps per position}] = 1 + \eta = 1,\ 3,\ 11$$

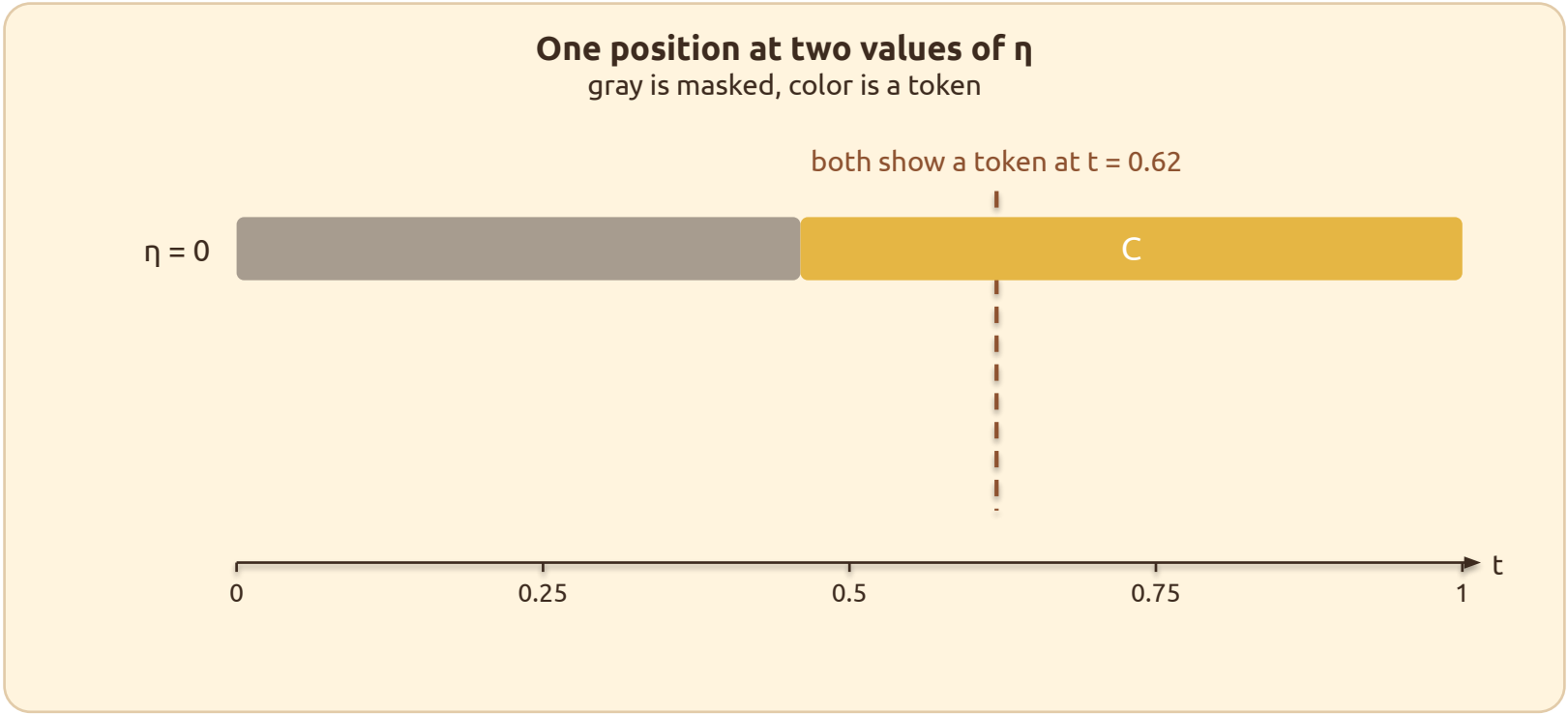
Across 200,000 simulated paths at each level, the masked fraction at t = 0.5 is 0.50 in all three cases. The final bases match the data distribution.

With more jumps, we can revisit an incorrect token while preserving the same marginals.

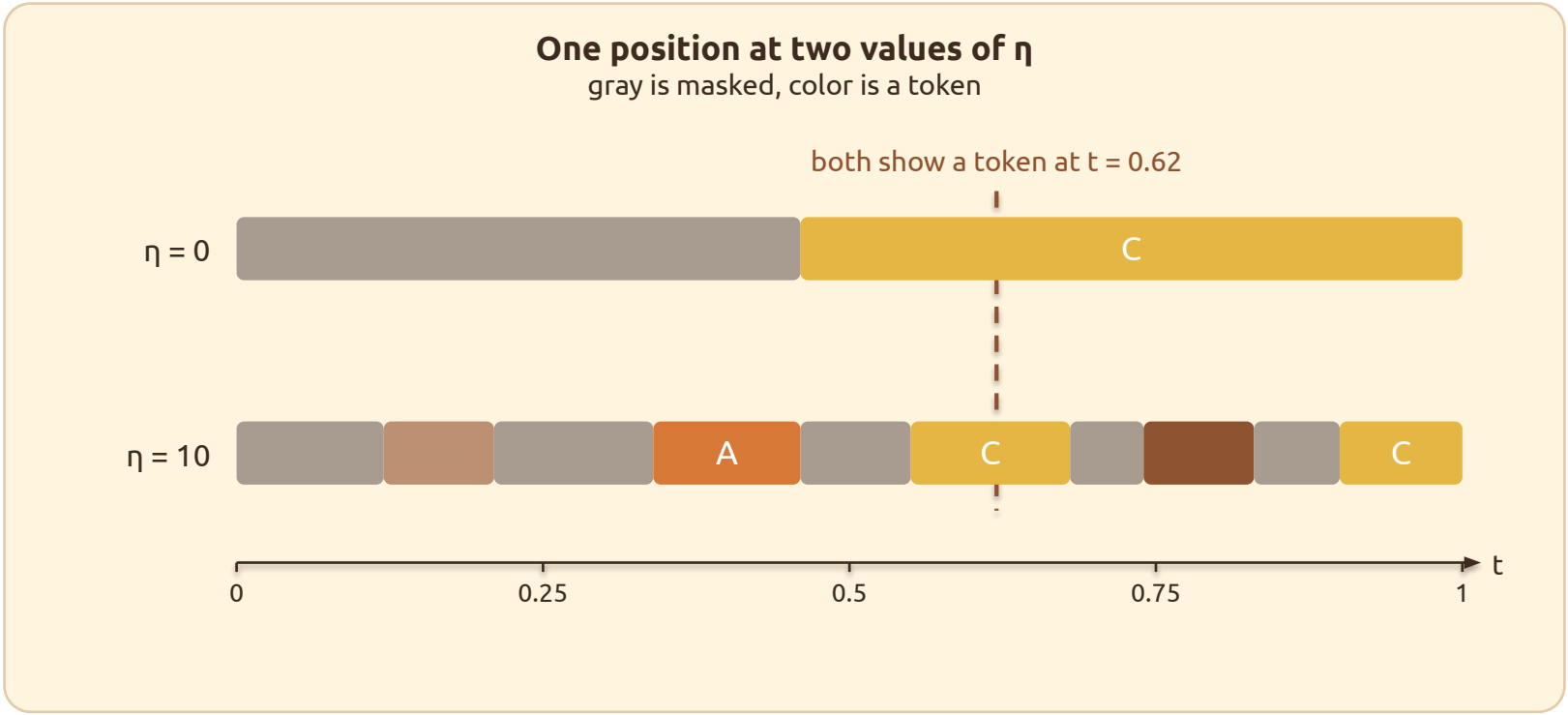
Here are two sampled paths for one position



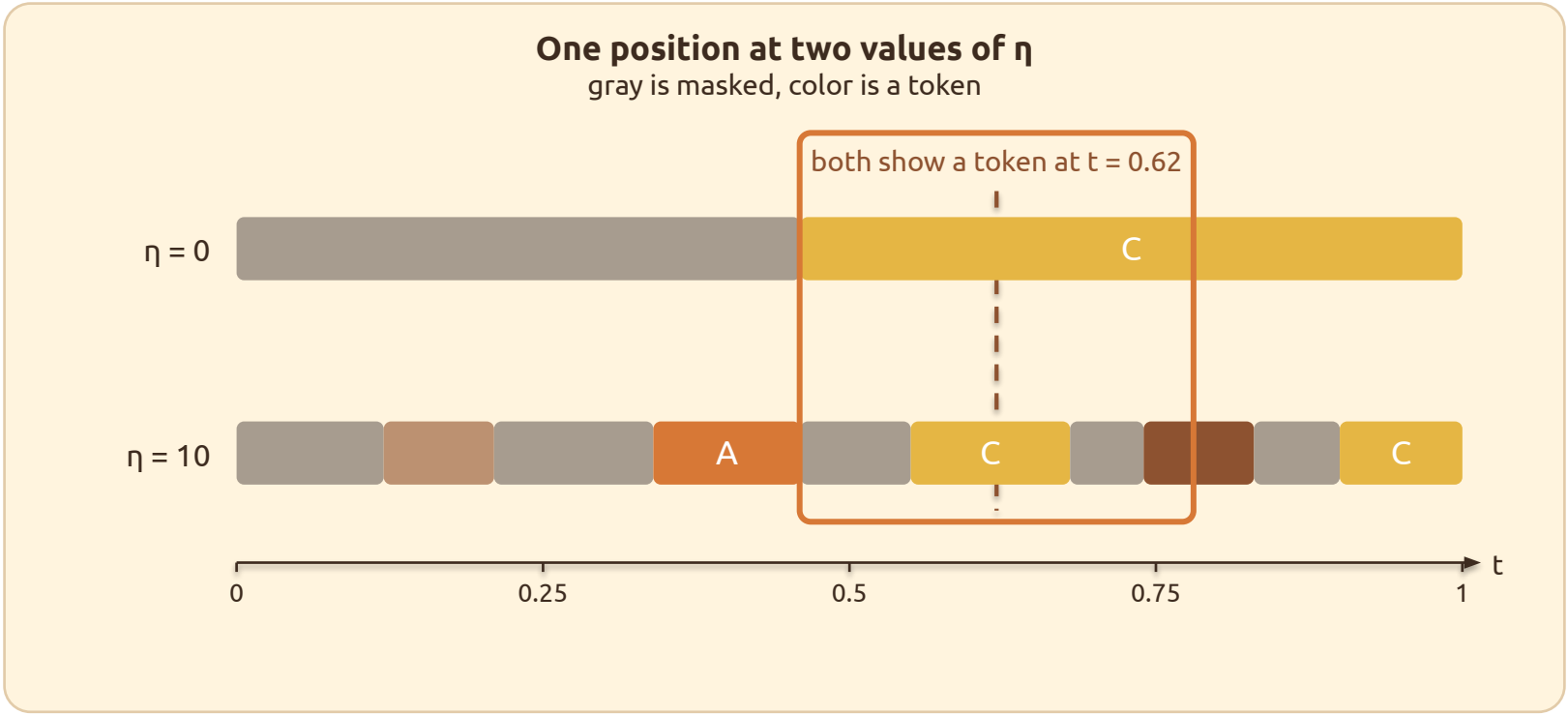
With η = 0, the position unmasks once and keeps its token



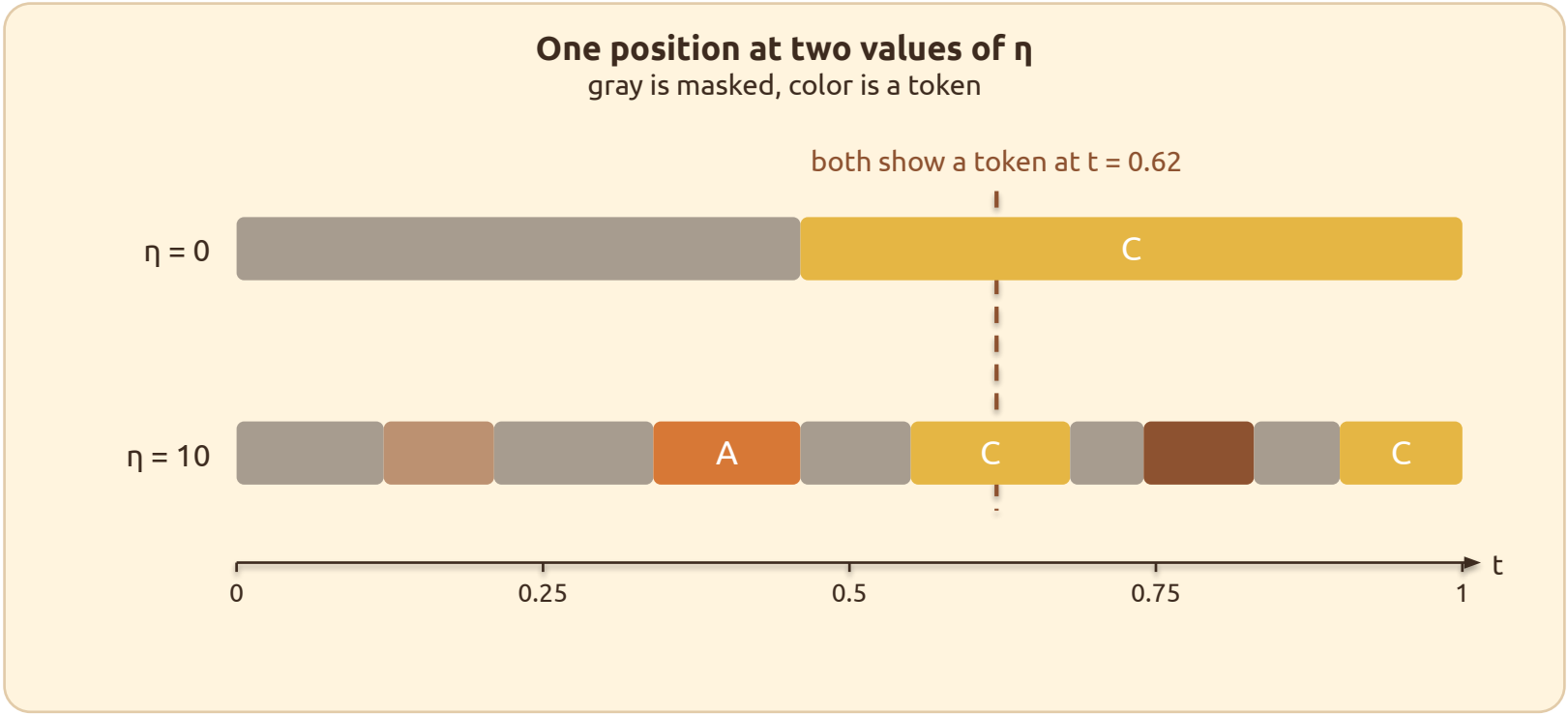
With η = 10, it unmasks and remasks many times before t = 1



At every time, the fraction of masked positions is the same for both



We change the sampled trajectories while preserving the probability path



Could we solve the master equation directly and use those rates to generate proteins?

Discrete Flow Models and Multiflow

Generative Flows on Discrete State-Spaces:
Enabling Multimodal Flows with Applications to Protein Co-Design

Andrew Campbell [*1] **Jason Yim** [*2] **Regina Barzilay** [2] **Tom Rainforth** [1] **Tommi Jaakkola** [2]

Campbell,..,Jaakkola. *ICML* (2024)

In Discrete Flow Models we solve the master equation directly

In Discrete Flow Models we solve the master equation directly

They start from any conditional path $p_t(\cdot \mid x_1)$ and solve the master equation directly.

In Discrete Flow Models we solve the master equation directly

They start from any conditional path $p_t(\cdot \mid x_1)$ and solve the master equation directly.

$$\boxed{u_t^\star(y, x \mid x_1) = \frac{\left[\, \dot{p}_t(y \mid x_1) - \dot{p}_t(x \mid x_1) \,\right]_+}{Z_t \; p_t(x \mid x_1)}, \qquad y \neq x}$$

In Discrete Flow Models we solve the master equation directly

They start from any conditional path $p_t(\cdot \mid x_1)$ and solve the master equation directly.

$$u_t^\star(y, x \mid x_1) = \frac{\left[\, \dot{p}_t(y \mid x_1) - \dot{p}_t(x \mid x_1) \,\right]_+}{Z_t \; p_t(x \mid x_1)}, \qquad y \neq x$$

Their rate from x to y equals the positive part of the rate of change of $p_t(y \mid x_1)$ minus the rate of change of $p_t(x \mid x_1)$, divided by Z_t, the number of states with positive probability, times the probability $p_t(x \mid x_1)$ of the current state.

In Discrete Flow Models we solve the master equation directly

They start from any conditional path $p_t(\cdot \mid x_1)$ and solve the master equation directly.

$$u_t^\star(y, x \mid x_1) = \frac{\left[\, \dot{p}_t(y \mid x_1) - \dot{p}_t(x \mid x_1) \,\right]_+}{Z_t \; p_t(x \mid x_1)}, \qquad y \neq x$$

Their rate from x to y equals the positive part of the rate of change of $p_t(y \mid x_1)$ minus the rate of change of $p_t(x \mid x_1)$, divided by Z_t, the number of states with positive probability, times the probability $p_t(x \mid x_1)$ of the current state.

Probability flows from x to y only when the probability of y must increase faster than that of x. We divide the required flux by the probability at x to obtain the rate.

Now compare that rate with our mixture velocity

Now compare that rate with our mixture velocity

For masked and uniform sources, $u^{\star}_{t}$ equals our mixture velocity. It minimizes the expected number of jumps, with one jump per masked position.

Now compare that rate with our mixture velocity

For masked and uniform sources, $u^{\star}_{t}$ equals our mixture velocity. It minimizes the expected number of jumps, with one jump per masked position.

In their main text, S, the vocabulary size, appears where Z_t belongs. With S, the master equation fails whenever $S > 2$.

Now compare that rate with our mixture velocity

For masked and uniform sources, $u^{\star}_{t}$ equals our mixture velocity. It minimizes the expected number of jumps, with one jump per masked position.

In their main text, S, the vocabulary size, appears where Z_t belongs. With S, the master equation fails whenever $S > 2$.

The η sampler we just built comes from the same paper, which has both the minimal-jump rate and the stochastic corrector.

Discrete Flow Models: moving probability

Consider a categorical flow on A and G with masses 0.75 and 0.25. Their desired derivatives are −1 and +1. Both states have positive mass.

Discrete Flow Models: moving probability

Consider a categorical flow on A and G with masses 0.75 and 0.25. Their desired derivatives are −1 and +1. Both states have positive mass.

$$u^*(\mathrm{G}, \mathrm{A}) = \frac{[1 - (-1)]_+}{2(0.75)} = \frac{4}{3}, \qquad u^*(\mathrm{A}, \mathrm{G}) = 0$$

Discrete Flow Models: moving probability

Consider a categorical flow on A and G with masses 0.75 and 0.25. Their desired derivatives are −1 and +1. Both states have positive mass.

$$u^*(\mathrm{G},\mathrm{A}) = \frac{[1-(-1)]_+}{2(0.75)} = \frac{4}{3}, \qquad u^*(\mathrm{A},\mathrm{G}) = 0$$

The rate moves probability from the shrinking A state to the growing G state.

Discrete Flow Models: moving probability

Consider a categorical flow on A and G with masses 0.75 and 0.25. Their desired derivatives are −1 and +1. Both states have positive mass.

$$u^*(\mathrm{G},\mathrm{A}) = \frac{[1-(-1)]_+}{2(0.75)} = \frac{4}{3}, \qquad u^*(\mathrm{A},\mathrm{G}) = 0$$

The rate moves probability from the shrinking A state to the growing G state.

$$\dot{p}(\mathrm{G}) = p(\mathrm{A})u^*(\mathrm{G},\mathrm{A}) - p(\mathrm{G})u^*(\mathrm{A},\mathrm{G}) = 0.75\frac{4}{3} = 1$$

Discrete Flow Models: moving probability

Consider a categorical flow on A and G with masses 0.75 and 0.25. Their desired derivatives are −1 and +1. Both states have positive mass.

$$u^*(\mathrm{G},\mathrm{A}) = \frac{[1-(-1)]_+}{2(0.75)} = \frac{4}{3}, \qquad u^*(\mathrm{A},\mathrm{G}) = 0$$

The rate moves probability from the shrinking A state to the growing G state.

$$\dot{p}(\mathrm{G}) = p(\mathrm{A})u^*(\mathrm{G},\mathrm{A}) - p(\mathrm{G})u^*(\mathrm{A},\mathrm{G}) = 0.75\frac{4}{3} = 1$$

Multiplying the rate by the source mass gives exactly the requested increase of one unit per unit time.

Discrete Flow Models: moving probability

Consider a categorical flow on A and G with masses 0.75 and 0.25. Their desired derivatives are −1 and +1. Both states have positive mass.

$$u^*(\mathrm{G},\mathrm{A}) = \frac{[1-(-1)]_+}{2(0.75)} = \frac{4}{3}, \qquad u^*(\mathrm{A},\mathrm{G}) = 0$$

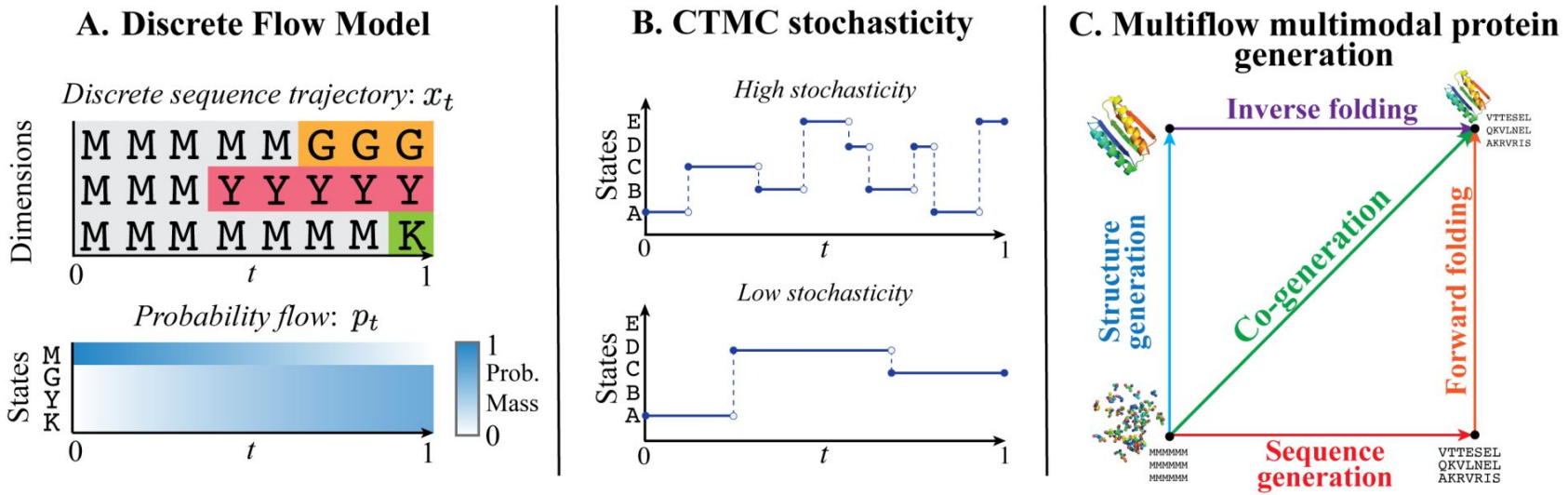
The rate moves probability from the shrinking A state to the growing G state.

$$\dot{p}(\mathrm{G}) = p(\mathrm{A})u^*(\mathrm{G},\mathrm{A}) - p(\mathrm{G})u^*(\mathrm{A},\mathrm{G}) = 0.75\frac{4}{3} = 1$$

Multiplying the rate by the source mass gives exactly the requested increase of one unit per unit time.

We can use the master equation to check how much probability the chosen rates transfer.

A masked path and stochastic corrector for protein generation



(A) A masked trajectory. (B) Different values of η change the trajectories while preserving the marginals.

Campbell,..,Jaakkola. *ICML* (2024)

In Multiflow we generate structure and sequence together

In Multiflow we generate structure and sequence together

Each residue has a position in space, an orientation, and an amino acid.

In Multiflow we generate structure and sequence together

Each residue has a position in space, an orientation, and an amino acid.

We use continuous flows for positions and orientations, with masked jumps for amino acids.

In Multiflow we generate structure and sequence together

Each residue has a position in space, an orientation, and an amino acid.

We use continuous flows for positions and orientations, with masked jumps for amino acids.

The conditional path factorizes over residues and modalities, but every velocity and rate depends on the whole noisy protein.

In Multiflow we generate structure and sequence together

Each residue has a position in space, an orientation, and an amino acid.

We use continuous flows for positions and orientations, with masked jumps for amino acids.

The conditional path factorizes over residues and modalities, but every velocity and rate depends on the whole noisy protein.

With separate times for structure and sequence, one model can fold, design sequences, or do both at once.

Now look at the refolding results for Multiflow

Table 3. Co-design results. Abbreviations: Designability (**DES.**), Diversity (**DIV.**), Novelty (**NOV.**). For Protpardelle, we report Co-design 1 as same numbers as PMPNN 1 since their co-design approach employs PMPNN. We note this is not co-generation since PMPNN is used while Multiflow explicitly learns co-generation without using PMPNN.

METHOD	CO-DESIGN 1			PMPNN 8			PMPNN 1		
	DES. (↑)	DIV. (↑)	NOV. (↓)	DES.	DIV.	NOV.	DES.	DIV.	NOV.
PROTPARDELLE	0.63*	38*	0.60*	0.90	47	**0.59**	0.63	38	**0.60**
PROTEINGENERATOR	0.37	35	0.69	0.89	75	0.65	0.78	64	0.66
RFDIFFUSION		N/A		0.87	**158**	0.63	0.66	111	0.64
MULTIFLOW	**0.86**	**143**	**0.61**	**0.99**	156	0.61	**0.88**	**143**	0.61
MULTIFLOW W/O DISTILLATION	0.42	72	0.62	0.89	126	0.62	0.71	101	0.63
MULTIFLOW W/O SEQUENCE		N/A		**0.99**	116	0.63	0.86	97	0.62

Campbell,..,Jaakkola. *ICML* (2024)

Generating structure and sequence together, 86% of its proteins refold to the designed structure

Table 3. Co-design results. Abbreviations: Designability (**DES.**), Diversity (**DIV.**), Novelty (**NOV.**). For Protpardelle, we report Co-design 1 as same numbers as PMPNN 1 since their co-design approach employs PMPNN. We note this is not co-generation since PMPNN is used while Multiflow explicitly learns co-generation without using PMPNN.

METHOD	CO-DESIGN 1			PMPNN 8			PMPNN 1		
	DES. (↑)	DIV. (↑)	NOV. (↓)	DES.	DIV.	NOV.	DES.	DIV.	NOV.
PROTPARDELLE	0.63*	38*	0.60*	0.90	47	**0.59**	0.63	38	**0.60**
PROTEINGENERATOR	0.37	35	0.69	0.89	75	0.65	0.78	64	0.66
RFDIFFUSION		N/A		0.87	**158**	0.63	0.66	111	0.64
MULTIFLOW	**0.86**	**143**	**0.61**	**0.99**	156	0.61	**0.88**	**143**	0.61
MULTIFLOW W/O DISTILLATION	0.42	72	0.62	0.89	126	0.62	0.71	101	0.63
MULTIFLOW W/O SEQUENCE		N/A		**0.99**	116	0.63	0.86	97	0.62

Campbell,..,Jaakkola. *ICML* (2024)

Without distilling better sequences into training, that fraction drops to 42%

Table 3. Co-design results. Abbreviations: Designability (**DES.**), Diversity (**DIV.**), Novelty (**NOV.**). For Protpardelle, we report Co-design 1 as same numbers as PMPNN 1 since their co-design approach employs PMPNN. We note this is not co-generation since PMPNN is used while Multiflow explicitly learns co-generation without using PMPNN.

METHOD	CO-DESIGN 1			PMPNN 8			PMPNN 1		
	DES. (↑)	DIV. (↑)	NOV. (↓)	DES.	DIV.	NOV.	DES.	DIV.	NOV.
PROTPARDELLE	0.63*	38*	0.60*	0.90	47	**0.59**	0.63	38	**0.60**
PROTEINGENERATOR	0.37	35	0.69	0.89	75	0.65	0.78	64	0.66
RFDIFFUSION		N/A		0.87	**158**	0.63	0.66	111	0.64
MULTIFLOW	**0.86**	**143**	**0.61**	**0.99**	156	0.61	**0.88**	**143**	0.61
MULTIFLOW W/O DISTILLATION	0.42	72	0.62	0.89	126	0.62	0.71	101	0.63
MULTIFLOW W/O SEQUENCE		N/A		**0.99**	116	0.63	0.86	97	0.62

Campbell,..,Jaakkola. *ICML* (2024)

Sampling used η = 20, so the stochastic corrector was part of the result

Table 3. Co-design results. Abbreviations: Designability (**DES.**), Diversity (**DIV.**), Novelty (**NOV.**). For Protpardelle, we report Co-design 1 as same numbers as PMPNN 1 since their co-design approach employs PMPNN. We note this is not co-generation since PMPNN is used while Multiflow explicitly learns co-generation without using PMPNN.

METHOD	CO-DESIGN 1			PMPNN 8			PMPNN 1		
	DES. (↑)	DIV. (↑)	NOV. (↓)	DES.	DIV.	NOV.	DES.	DIV.	NOV.
PROTPARDELLE	0.63*	38*	0.60*	0.90	47	**0.59**	0.63	38	**0.60**
PROTEINGENERATOR	0.37	35	0.69	0.89	75	0.65	0.78	64	0.66
RFDIFFUSION		N/A		0.87	**158**	0.63	0.66	111	0.64
MULTIFLOW	**0.86**	**143**	**0.61**	**0.99**	156	0.61	**0.88**	**143**	0.61
MULTIFLOW W/O DISTILLATION	0.42	72	0.62	0.89	126	0.62	0.71	101	0.63
MULTIFLOW W/O SEQUENCE		N/A		**0.99**	116	0.63	0.86	97	0.62

Campbell,..,Jaakkola. *ICML* (2024)

If many rates follow the same path, which ones move probability with the least energy?

Flow Matching with General Discrete Paths: A Kinetic-Optimal Perspective

FLOW MATCHING WITH GENERAL DISCRETE PATHS: A KINETIC-OPTIMAL PERSPECTIVE

Neta Shaul[1,†], **Itai Gat**[2], **Marton Havasi**[2], **Daniel Severo**[2], **Anuroop Sriram**[2], **Peter Holderrieth**[3,†], **Brian Karrer**[2], **Yaron Lipman**[2], **Ricky T. Q. Chen**[2]
[1]Weizmann Institute of Science, [2]Meta FAIR, [3]MIT CSAIL
[†]Work done during internship at Meta FAIR

Shaul,..,Chen. *ICLR* (2025)

Now write the flux in place of the velocity

Now write the flux in place of the velocity

$$\boxed{j_t(y, x) = u_t(y, x)\, p_t(x)}$$

Now write the flux in place of the velocity

$$\boxed{j_t(y, x) = u_t(y, x)\, p_t(x)}$$

The flux $j_t(y, x)$ from x to y equals the velocity $u_t(y, x)$ times the probability $p_t(x)$ of the current state.

Now write the flux in place of the velocity

$$\boxed{j_t(y, x) = u_t(y, x)\, p_t(x)}$$

The flux $j_t(y, x)$ from x to y equals the velocity $u_t(y, x)$ times the probability $p_t(x)$ of the current state.

The master equation is linear in the flux, so the constraint on j is a linear one.

Among all fluxes, choose the one with least energy

Among all fluxes, choose the one with least energy

$$\min_{j} \int_0^1 \sum_{y \neq x} \frac{w_t(y,x)}{p_t(x)} \, j_t(y,x)^2 \, dt \quad \text{subject to the master equation and } j \geq 0$$

Among all fluxes, choose the one with least energy

$$\min_{j} \int_0^1 \sum_{y \neq x} \frac{w_t(y,x)}{p_t(x)} \, j_t(y,x)^2 \, dt \quad \text{subject to the master equation and } j \geq 0$$

Among all nonnegative fluxes that satisfy the master equation, choose the one that minimizes the integral over time of the sum over edges of a weight $w_t(y, x)$ divided by $p_t(x)$, times the squared flux.

Among all fluxes, choose the one with least energy

$$\min_{j} \int_0^1 \sum_{y \neq x} \frac{w_t(y,x)}{p_t(x)} \, j_t(y,x)^2 \, dt \quad \text{subject to the master equation and } j \geq 0$$

Among all nonnegative fluxes that satisfy the master equation, choose the one that minimizes the integral over time of the sum over edges of a weight $w_t(y, x)$ divided by $p_t(x)$, times the squared flux.

In velocity notation, the cost is the sum of $w_t(y, x)\, u_t(y, x)^2\, p_t(x)$. With w equal to one, this is the discrete counterpart of expected squared speed in continuous optimal transport.

Among all fluxes, choose the one with least energy

$$\min_{j} \int_0^1 \sum_{y \neq x} \frac{w_t(y,x)}{p_t(x)} \, j_t(y,x)^2 \, dt \quad \text{subject to the master equation and } j \geq 0$$

Among all nonnegative fluxes that satisfy the master equation, choose the one that minimizes the integral over time of the sum over edges of a weight $w_t(y, x)$ divided by $p_t(x)$, times the squared flux.

In velocity notation, the cost is the sum of $w_t(y, x)\, u_t(y, x)^2\, p_t(x)$. With w equal to one, this is the discrete counterpart of expected squared speed in continuous optimal transport.

The constraints are linear and the cost is convex, so the problem has a closed-form answer.

Among all fluxes, choose the one with least energy

$$\min_{j} \int_0^1 \sum_{y \neq x} \frac{w_t(y,x)}{p_t(x)} \, j_t(y,x)^2 \, dt \quad \text{subject to the master equation and } j \geq 0$$

Among all nonnegative fluxes that satisfy the master equation, choose the one that minimizes the integral over time of the sum over edges of a weight $w_t(y, x)$ divided by $p_t(x)$, times the squared flux.

In velocity notation, the cost is the sum of $w_t(y, x)\, u_t(y, x)^2\, p_t(x)$. With w equal to one, this is the discrete counterpart of expected squared speed in continuous optimal transport.

Doubling a flux makes its squared cost four times larger, discouraging unnecessary rapid jumps.

Now choose a weight and solve for the optimal velocity

Now choose a weight and solve for the optimal velocity

With the weight $w_t(y, x) = 1/p_t(y)$, the optimal velocity is

$$\boxed{u_t^\star(y, x) = p_t(y) \left[\frac{d}{dt} \log p_t(y) - \frac{d}{dt} \log p_t(x) \right]_+}$$

Now choose a weight and solve for the optimal velocity

With the weight $w_t(y, x) = 1/p_t(y)$, the optimal velocity is

$$u_t^\star(y, x) = p_t(y) \left[\frac{d}{dt} \log p_t(y) - \frac{d}{dt} \log p_t(x) \right]_+$$

The kinetic-optimal velocity from x to y equals the probability $p_t(y)$ times the positive part of the growth rate of log $p_t(y)$ minus the growth rate of log $p_t(x)$.

Now choose a weight and solve for the optimal velocity

With the weight $w_t(y, x) = 1/p_t(y)$, the optimal velocity is

$$u_t^\star(y, x) = p_t(y) \left[\frac{d}{dt} \log p_t(y) - \frac{d}{dt} \log p_t(x) \right]_+$$

The kinetic-optimal velocity from x to y equals the probability $p_t(y)$ times the positive part of the growth rate of log $p_t(y)$ minus the growth rate of log $p_t(x)$.

Probability moves from x to y only when y is growing faster, relative to its size, than x.

Now choose a weight and solve for the optimal velocity

With the weight $w_t(y, x) = 1/p_t(y)$, the optimal velocity is

$$u_t^\star(y, x) = p_t(y) \left[\frac{d}{dt} \log p_t(y) - \frac{d}{dt} \log p_t(x) \right]_+$$

The kinetic-optimal velocity from x to y equals the probability $p_t(y)$ times the positive part of the growth rate of log $p_t(y)$ minus the growth rate of log $p_t(x)$.

Probability moves from x to y only when y is growing faster, relative to its size, than x.

For every mixture path, with any source, this is the mixture velocity of Discrete Flow Matching.

Now choose a weight and solve for the optimal velocity

With the weight $w_t(y, x) = 1/p_t(y)$, the optimal velocity is

$$u_t^\star(y, x) = p_t(y) \left[\frac{d}{dt} \log p_t(y) - \frac{d}{dt} \log p_t(x) \right]_+$$

The kinetic-optimal velocity from x to y equals the probability $p_t(y)$ times the positive part of the growth rate of log $p_t(y)$ minus the growth rate of log $p_t(x)$.

Probability moves from x to y only when y is growing faster, relative to its size, than x.

For every mixture path, with any source, this is the mixture velocity of Discrete Flow Matching.

It also vanishes when $p_t(x)$ or $p_t(y)$ is zero, so it remains valid when some states have zero probability.